Chapitre 1: Prérequis d'algèbre linéaire

Définition d'une matrice symétrique définie et positive

On définie A une matrice symétrique de taille $N \times N$ si elle vérifie:

$$\forall x \in \mathbb{R}^N, x^T A x > 0 \quad (1)$$

$$A^T = A \quad (2)$$

Théorème (décomposition d'une matrice symétrique en matrices par blocs) :

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix}$$

De surcroît, $A_{1,1}$ et $A_{2,2}$ sont symétriques définies et positives.

Démonstration :

Notations

On pose :

$$A = (a_{i,j})_{(i,j)\in[1,n]^2}$$

$$A_{1,1} = (a_{i,j})_{(i,j)\in[1,k]^2}$$

$$A_{1,2} = (a_{i,j})_{(i,j)\in[1,k]\times[k+1,n]}$$

$$A_{2,1} = (a_{i,j})_{(i,j)\in[k+1,n]\times[1,k]}$$

$$A_{2,2} = (a_{i,j})_{(i,j)\in[k+1,n]^2}$$

D'où la matrice $A \in \mathcal{M}_n(\mathbb{R})$ s'écrit aussi :

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

De même, la symétrie de A impose que pour tous sous-ensembles d'indices $I, J \subset [\![1, n]\!]$:

$$(a_{i,j})_{(i,j)\in I\times J} = (a_{j,i})_{(i,j)\in J\times I}$$

Réécriture de $A_{2,1}$:

Par symétrie de A, on sait que :

$$A_{2,1} = (a_{i,j})_{(i,j)\in[k+1,n]\times[1,k]} = (a_{j,i})_{(j,i)\in[1,k]\times[k+1,n]} = A_{1,2}^T$$

D'où:

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix}$$

Caractère défini positif des blocs diagonaux $A_{1,1}$ et $A_{2,2}$

Soit un vecteur $X = \begin{pmatrix} 0_{\mathbb{R}^k} \\ x_1 \end{pmatrix} \in \mathbb{R}^n$, où $x_1 \in \mathbb{R}^{n-k}$ est un vecteur quelconque non nul ($x_1 \neq 0$).Puisque la matrice A est symétrique et définie positive, on a par définition pour tout vecteur non nul X :

$$X^T A X > 0$$

En effectuant le produit matriciel par blocs avec X :

$$\begin{aligned} X^T A X &= \begin{pmatrix} 0_{\mathbb{R}^k}^T & x_1^T \end{pmatrix} \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix} \begin{pmatrix} 0_{\mathbb{R}^k} \\ x_1 \end{pmatrix} \\ &= \begin{pmatrix} 0_{\mathbb{R}^k}^T & x_1^T \end{pmatrix} \begin{pmatrix} A_{1,2} x_1 \\ A_{2,2} x_1 \end{pmatrix} \\ &= 0_{\mathbb{R}^k}^T (A_{1,2} x_1) + x_1^T A_{2,2} x_1 \\ &= x_1^T A_{2,2} x_1 \end{aligned}$$

Comme A est définie positive, $X^T A X > 0$ pour tout $X \neq 0$, ce qui implique que :

$$x_1^T A_{2,2} x_1 > 0 \quad \text{pour tout } x_1 \neq 0$$

Le bloc $A_{2,2}$ est donc symétrique défini positif.

(On procède de manière strictement analogue pour $A_{1,1}$ en prenant le vecteur $Y = \begin{pmatrix} y_1 \\ 0_{\mathbb{R}^{n-k}} \end{pmatrix}$ avec $y_1 \in \mathbb{R}^k$ non nul, ce qui donne $Y^T A Y = y_1^T A_{1,1} y_1 > 0$.)

Caractère symétrique des blocs diagonaux $A_{1,1}$ et $A_{2,2}$

Puisque la matrice globale A est symétrique, on a par définition $A = A^T$.

En écrivant cette égalité sous forme de blocs :

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} = \begin{pmatrix} A_{1,1}^T & A_{2,1}^T \\ A_{1,2}^T & A_{2,2}^T \end{pmatrix}$$

Par identification terme à terme des sous-matrices (ou blocs) :

Pour le bloc diagonal supérieur : $A_{1,1} = A_{1,1}^T$, ce qui prouve que $A_{1,1}$ est symétrique.

Pour le bloc diagonal inférieur : $A_{2,2} = A_{2,2}^T$, ce qui prouve que $A_{2,2}$ est symétrique.

Théorème (Complément de Schur et décomposition par blocs) :

Soit A une matrice symétrique par blocs de la forme :

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix}$$

où $A_{1,1}$ et $A_{2,2}$ sont des blocs carrés inversibles (et par conséquent symétriques définis et positifs).

La matrice A peut être factorisée par blocs à l'aide du **complément de Schur** de $A_{1,1}$ (noté S_1) ou de $A_{2,2}$ (noté S_2).

1. Factorisation via le complément de Schur de $A_{1,1}$

Si $A_{1,1}$ est inversible, on peut décomposer A sous la forme :

$$A = \begin{pmatrix} I & 0 \\ A_{1,2}^T A_{1,1}^{-1} & I \end{pmatrix} \begin{pmatrix} A_{1,1} & 0 \\ 0 & S_1 \end{pmatrix} \begin{pmatrix} I & A_{1,1}^{-1} A_{1,2} \\ 0 & I \end{pmatrix}$$

Où S_1 est le complément de Schur de $A_{1,1}$, défini par :

$$S_1 = A_{2,2} - A_{1,2}^T A_{1,1}^{-1} A_{1,2}$$

2. Équivalence pour le caractère défini positif

Dans ce cadre, la matrice A est symétrique définie positive si et seulement si :

- Le bloc diagonal $A_{1,1}$ est symétrique défini positif.
- Son complément de Schur S_1 est également symétrique défini positif.

(De manière symétrique, en inversant les rôles, A est définie positive si et seulement si $A_{2,2}$ et son complément de Schur $S_2 = A_{1,1} - A_{1,2}A_{2,2}^{-1}A_{1,2}^T$ le sont également).

Démonstration du Théorème du Complément de Schur

1. Démonstration de la factorisation par blocs

Il s'agit de vérifier par le calcul que le produit des trois matrices de la décomposition redonne exactement la matrice A.

Posons la factorisation avec le complément de Schur $S_1 = A_{2,2} - A_{1,2}^T A_{1,1}^{-1} A_{1,2}$:

$$\begin{pmatrix} I & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ A_{1,2}^T A_{1,1}^{-1} & I \end{pmatrix} \begin{pmatrix} A_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} \begin{pmatrix} I & A_{1,1}^{-1}A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & I \end{pmatrix}$$

Étape 1 : Multiplication des deux derniers blocs de droite

$$\begin{pmatrix} A_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} \begin{pmatrix} I & A_{1,1}^{-1}A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & I \end{pmatrix} = \begin{pmatrix} A_{1,1} \cdot I + 0 \cdot 0 & A_{1,1}(A_{1,1}^{-1}A_{1,2}) + 0 \cdot I \\ 0 \cdot I + S_1 \cdot 0 & 0 \cdot (A_{1,1}^{-1}A_{1,2}) + S_1 \cdot I \end{pmatrix} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix}$$

Étape 2 : Multiplication avec le premier bloc de gauche Multiplions maintenant le résultat précédent par la première matrice :

$$\begin{pmatrix} I & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ A_{1,2}^T A_{1,1}^{-1} & I \end{pmatrix} \begin{pmatrix} A_{1,1} & A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} = \begin{pmatrix} I \cdot A_{1,1} + 0 \cdot 0 & I \cdot A_{1,2} + 0 \cdot S_1 \\ (A_{1,2}^T A_{1,1}^{-1})A_{1,1} + I \cdot 0 & (A_{1,2}^T A_{1,1}^{-1})A_{1,2} + I \cdot S_1 \end{pmatrix}$$

Simplifions chaque bloc terme à terme :

- **Haut-gauche :** $A_{1,1}$
- **Haut-droite :** $A_{1,2}$
- **Bas-gauche :** $A_{1,2}^T A_{1,1}^{-1} A_{1,1} = A_{1,2}^T$
- **Bas-droite :** $A_{1,2}^T A_{1,1}^{-1} A_{1,2} + S_1$

En remplaçant S_1 par sa définition $S_1 = A_{2,2} - A_{1,2}^T A_{1,1}^{-1} A_{1,2}$, le bloc bas-droite devient :

$$A_{1,2}^T A_{1,1}^{-1} A_{1,2} + \left(A_{2,2} - A_{1,2}^T A_{1,1}^{-1} A_{1,2} \right) = A_{2,2}$$

On retombe bien sur la matrice initiale A :

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix}$$

La factorisation est donc rigoureusement démontrée.

Notons que si l'on pose $P = \begin{pmatrix} I & A_{1,1}^{-1} A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & I \end{pmatrix}$, cette relation s'écrit sous forme de congruence :
$A = P^T \begin{pmatrix} A_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} P.$

---

2. Démonstration de l'équivalence pour le caractère défini positif

Puisque la matrice de passage P est inversible, étudions la forme quadratique associée à A pour un vecteur quelconque
$X = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^n.$

Effectuons le changement de variable $Y = PX = \begin{pmatrix} I & A_{1,1}^{-1} A_{1,2} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & I \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$, avec $y_1 = x_1 + A_{1,1}^{-1} A_{1,2} x_2$ et $y_2 = x_2$
. Comme P est une bijection, $X \neq 0_{\mathbb{R}^n} \iff Y \neq 0_{\mathbb{R}^n}$.

En remplaçant A par sa forme factorisée :

$$X^T A X = (PX)^T \begin{pmatrix} A_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} (PX) = Y^T \begin{pmatrix} A_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & S_1 \end{pmatrix} Y = y_1^T A_{1,1} y_1 + y_2^T S_1 y_2$$

Sens $(\Rightarrow)$: Si A est définie positive

1. On sait déjà par les propriétés des blocs diagonaux que $A_{1,1}$ est définie positive.
2. Pour montrer que S_1 est défini positif, soit un vecteur non nul $x_2 \in \mathbb{R}^{n-k} \setminus \{0_{\mathbb{R}^{n-k}}\}$. Choisissons le vecteur X particulier tel que $y_1 = 0_{\mathbb{R}^k}$ (c'est-à-dire en posant $x_1 = -A_{1,1}^{-1}A_{1,2}x_2$). Le vecteur global X est non nul car $x_2 \neq 0_{\mathbb{R}^{n-k}}$. Dès lors :

$$X^T AX = 0_{\mathbb{R}^k}^T A_{1,1} 0_{\mathbb{R}^k} + x_2^T S_1 x_2 = x_2^T S_1 x_2$$

Puisque A est définie positive, $X^T AX > 0$, ce qui implique que $x_2^T S_1 x_2 > 0$ pour tout $x_2 \neq 0_{\mathbb{R}^{n-k}}$. S_1 **est donc symétrique défini positif.**

Sens $(\Leftarrow)$: Si $A_{1,1}$ et S_1 sont définis positifs

Soit un vecteur quelconque $X \neq 0_{\mathbb{R}^n}$, ce qui implique que $Y = PX \neq 0_{\mathbb{R}^n}$ (donc y_1 et y_2 ne sont pas simultanément nuls). Puisque $A_{1,1}$ et S_1 sont définis positifs :

- Si $y_1 \neq 0_{\mathbb{R}^k}$, alors $y_1^T A_{1,1} y_1 > 0$ (et $y_2^T S_1 y_2 \geq 0$).
- Si $y_2 \neq 0_{\mathbb{R}^{n-k}}$, alors $y_2^T S_1 y_2 > 0$ (et $y_1^T A_{1,1} y_1 \geq 0$).

Dans tous les cas, la somme est strictement positive :

$$X^T AX = y_1^T A_{1,1} y_1 + y_2^T S_1 y_2 > 0$$

La matrice A est donc **symétrique définie positive**.

(Remarque : Par stricte symétrie des rôles et permutation des blocs, cette démonstration s'applique exactement de la même manière pour le complément de Schur $S_2 = A_{1,1} - A_{1,2}A_{2,2}^{-1}A_{1,2}^T$ associé au bloc $A_{2,2}$, en utilisant la factorisation symétrique par le bas et un changement de variable adapté).

Théorème (Décomposition de Cholesky)

Soit $A \in \mathcal{M}_n(\mathbb{R})$ une matrice symétrique définie positive.

Il existe une unique matrice triangulaire inférieure $L \in \mathcal{M}_n(\mathbb{R})$ dont tous les éléments diagonaux sont strictement positifs, telle que :

$$A = LL^T$$

Démonstration (par récurrence et par blocs via le complément de Schur)

Nous procédons par récurrence sur la dimension n de la matrice A.

1. Initialisation (pour $n = 1$)

Si $A = (a_{1,1})$ est une matrice 1×1 symétrique définie positive, alors $a_{1,1} > 0$. Il suffit de poser $L = (\sqrt{a_{1,1}})$, qui est bien une matrice triangulaire inférieure avec un élément diagonal strictement positif, et l'on a bien $LL^T = (\sqrt{a_{1,1}})(\sqrt{a_{1,1}}) = (a_{1,1}) = A$.

---

2. Hérédité et Construction par Blocs

Supposons le théorème vrai pour toute matrice symétrique définie positive de dimension strictement inférieure à n.

Considérons une matrice $A \in \mathcal{M}_n(\mathbb{R})$ symétrique définie positive. Découpons-la en blocs de taille $k \times k$ et $(n-k) \times (n-k)$ (avec par exemple $k = 1$) :

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{1,2}^T & A_{2,2} \end{pmatrix}$$

D'après les propriétés établies précédemment, le bloc diagonal $A_{1,1}$ est lui-même symétrique défini positif (de dimension $k < n$).

Par hypothèse de récurrence, $A_{1,1}$ admet une unique décomposition de Cholesky :

$$A_{1,1} = L_{1,1} L_{1,1}^T$$

où $L_{1,1}$ est une matrice triangulaire inférieure à diagonale strictement positive.

Cherchons à exprimer la matrice de Cholesky globale L sous la forme d'une matrice triangulaire inférieure par blocs :

$$L = \begin{pmatrix} L_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ L_{2,1} & L_{2,2} \end{pmatrix}$$

En effectuant le produit LL^T :

$$LL^T = \begin{pmatrix} L_{1,1} & 0_{\mathcal{M}_{k,n-k}(\mathbb{R})} \\ L_{2,1} & L_{2,2} \end{pmatrix} \begin{pmatrix} L_{1,1}^T & L_{2,1}^T \\ 0_{\mathcal{M}_{n-k,k}(\mathbb{R})} & L_{2,2}^T \end{pmatrix} = \begin{pmatrix} L_{1,1}L_{1,1}^T & L_{1,1}L_{2,1}^T \\ L_{2,1}L_{1,1}^T & L_{2,1}L_{2,1}^T + L_{2,2}L_{2,2}^T \end{pmatrix}$$

Par identification terme à terme avec la matrice A :

1. **Premier bloc diagonal :**

$$L_{1,1}L_{1,1}^T = A_{1,1}$$

(ce qui est résolu par l'hypothèse de récurrence).

2. **Bloc extra-diagonal :**

$$L_{1,1}L_{2,1}^T = A_{1,2} \implies L_{2,1}^T = L_{1,1}^{-1}A_{1,2} \implies L_{2,1} = A_{1,2}^T L_{1,1}^{-T}$$

Puisque $L_{1,1}$ est inversible (ses éléments diagonaux étant strictement positifs), le bloc $L_{2,1}$ est ainsi entièrement et de manière unique déterminé.

3. **Second bloc diagonal :**

$$L_{2,1}L_{2,1}^T + L_{2,2}L_{2,2}^T = A_{2,2}$$

En isolant le terme en $L_{2,2}$:

$$L_{2,2}L_{2,2}^T = A_{2,2} - L_{2,1}L_{2,1}^T$$

En remplaçant $L_{2,1}$ par son expression :

$$L_{2,1}L_{2,1}^T = (A_{1,2}^T L_{1,1}^{-T})(L_{1,1}^{-1}A_{1,2}) = A_{1,2}^T (L_{1,1}L_{1,1}^T)^{-1} A_{1,2} = A_{1,2}^T A_{1,1}^{-1} A_{1,2}$$

On reconduit ainsi exactement le **complément de Schur** de $A_{1,1}$:

$$L_{2,2}L_{2,2}^T = A_{2,2} - A_{1,2}^T A_{1,1}^{-1} A_{1,2} = S_1$$

Puisque la matrice globale A est définie positive, son complément de Schur S_1 est lui aussi symétrique défini positif (de dimension $n-k<n$). Par l'hypothèse de récurrence, S_1 admet donc une unique décomposition de Cholesky :

$$S_1 = L_{2,2}L_{2,2}^T$$

où $L_{2,2}$ est triangulaire inférieure à diagonale strictement positive.

Conclusion

Par construction unique des blocs $L_{1,1}$, $L_{2,1}$ et $L_{2,2}$, la matrice L est triangulaire inférieure, ses coefficients diagonaux (issus de ceux de $L_{1,1}$ et $L_{2,2}$) sont strictement positifs, et l'on a bien :

$$A = LL^T$$

La décomposition de Cholesky est ainsi démontrée par récurrence.

Théorème (Formules explicites des coefficients de Cholesky)

Soit $A \in \mathcal{M}_n(\mathbb{R})$ une matrice symétrique définie positive, dont les coefficients sont notés a_{ij} pour $1 \leq i, j \leq n$.

Il existe une unique matrice triangulaire inférieure $L \in \mathcal{M}_n(\mathbb{R})$, de coefficients l_{ij} (avec $l_{ij} = 0$ pour tout $j > i$), telle que $A = LL^T$, et dont les coefficients diagonaux sont strictement positifs ($l_{ii} > 0$ pour tout $1 \leq i \leq n$).

Les coefficients l_{ij} de la matrice L sont déterminés de manière unique et séquentielle par les relations suivantes :

1. Pour les éléments diagonaux ($i = j$) :

Pour chaque ligne i allant de 1 à n :

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}$$

(Par convention, pour $i = 1$, la somme vide est égale à 0, ce qui donne $l_{11} = \sqrt{a_{11}}$).

2. Pour les éléments sous-diagonaux ($i > j$) :

Pour chaque ligne i et colonne j telles que $1 \leq j < i \leq n$:

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$$

(Par convention, pour $j = 1$, la somme vide est également nulle).

3. Pour les éléments au-dessus de la diagonale ($i < j$) :

Puisque L est une matrice triangulaire inférieure :

$$l_{ij} = 0 \quad (\text{ou sous forme de matrice nulle de dimension adéquate})$$

Démonstration de l'unicité par identification (Aperçu) :

En développant le produit matriciel terme à terme de l'égalité $A = LL^T$, on a pour tout couple (i, j) :

$$a_{ij} = \sum_{k=1}^{n} l_{ik} (L^T)_{kj} = \sum_{k=1}^{n} l_{ik} l_{jk}$$

Puisque $l_{ik} = 0$ pour $k > i$ et $l_{jk} = 0$ pour $k > j$, la somme s'arrête au maximum à $\min(i, j)$.

- **Cas $i = j$:** $a_{ii} = \sum_{k=1}^{i} l_{ik}^2 = \left(\sum_{k=1}^{i-1} l_{ik}^2 \right) + l_{ii}^2$. En isolant l_{ii} et en posant $l_{ii} > 0$, on obtient la formule diagonale (le terme sous la racine est strictement positif car A est définie positive).
- **Cas $i > j$ (Au-dessous de la diagonale de L):** $a_{ij} = \sum_{k=1}^{j} l_{ik} l_{jk} = \left(\sum_{k=1}^{j-1} l_{ik} l_{jk} \right) + l_{ij} l_{jj}$. Comme $l_{jj} > 0$, on peut diviser pour isoler l_{ij}, ce qui donne la formule sous-diagonale.

- **Cas $i < j$ (Au-dessus de la diagonale de L):** Puisque L est triangulaire inférieure, on a par définition $l_{ik} = 0$ pour tout $k > i$. Comme $i < j$, pour tout indice de sommation $k \in \{1, \ldots, n\}$, on a nécessairement soit $k \leq i < j$ (donc l_{ik} peut être non nul, mais alors $l_{jk} = 0$ car $k < j$ et L est triangulaire inférieure), soit $k > i$ (ce qui annule l_{ik}).

Note: Cette construction récurrente (ligne par ligne, de gauche à droite sur chaque ligne) prouve à la fois l'existence et l'unicité de la matrice L.

Implémentation de la décomposition de Cholesky en Python

In [79]:
```python
import numpy as np

def cholesky(A : np.ndarray) -> np.ndarray:
    n, m = np.shape(A)

    if n != m:
        raise ValueError(
        f"Error : The dimensions do not match (given dimensions : {n}x{m})."
    )

    if not np.allclose(A,A.T):
        raise ValueError(
        f"Error : The given matrix is not symetrical."
    )

    L = np.zeros((int(n),int(n)))

    for i in range(n):
        for j in range(i+1) : # Thus i≤j
            if i == j: #Diagonal cases (i = j)
                L [i,i] = np.sqrt(A[i,i] - np.sum(L[i,:i]**2))
            if j < i: #Sub-diagonal cases (j < i)
                L[i,j] = (A[i,j] - np.sum(L[i,:j]*L[j,:j]))/L[j,j]
    return L
```

In [78]:
```
A = np.array([
    [4,2,2],
    [2,4,2],
    [2,2,4]
])

print(cholesky(A))

print("==="*15)
print("==="*15)
print("==="*15)

B = np.array([
    [1,1,1,1],
    [1,2,1,1],
    [1,1,3,1],
    [1,1,1,4]
])

print(cholesky(B))
```

```
[[2.         0.         0.        ]
 [1.         1.73205081 0.        ]
 [1.         0.57735027 1.63299316]]
=============================================
=============================================
=============================================
[[1.         0.         0.         0.        ]
 [1.         1.         0.         0.        ]
 [1.         0.         1.41421356 0.        ]
 [1.         0.         0.         1.73205081]]
```

In [81]:
```
print(cholesky(A) @ cholesky(A).T)

print("==="*15)
print("==="*15)
print("==="*15)

print(cholesky(B) @ cholesky(B).T)
```

```
[[4. 2. 2.]
 [2. 4. 2.]
 [2. 2. 4.]]
==========================================
==========================================
==========================================
[[1. 1. 1. 1.]
 [1. 2. 1. 1.]
 [1. 1. 3. 1.]
 [1. 1. 1. 4.]]
```

Pour une utilisation plus simple, on utilisera directement la méthode np.linalg.cholesky du module NumPy:

In [84]:
```
print(np.linalg.cholesky(A))

print("==="*15)
print("==="*15)
print("==="*15)

print(np.linalg.cholesky(B))
```

```
[[2.         0.         0.        ]
 [1.         1.73205081 0.        ]
 [1.         0.57735027 1.63299316]]
==========================================
==========================================
==========================================
[[1.         0.         0.         0.        ]
 [1.         1.         0.         0.        ]
 [1.         0.         1.41421356 0.        ]
 [1.         0.         0.         1.73205081]]
```

Chapitre 2: Prérequis sur les vecteurs aléatoires

Définition d'un vecteur aléatoire

Un vecteur aléatoire (ou variable aléatoire multivariée) de dimension p est une application X définie sur l'univers Ω à valeurs dans l'espace vectoriel $\mathbb{R}^p$:

$$X : \Omega \longrightarrow \mathbb{R}^p, \quad \text{avec} \quad \omega \longmapsto X(\omega) = \begin{pmatrix} X_1(\omega) \\ X_2(\omega) \\ \vdots \\ X_p(\omega) \end{pmatrix}$$

Définition de l'espérance d'un vecteur aléatoire

Soit X un vecteur aléatoire de dimension p dont chaque composante X_i est intégrable, c'est-à-dire qu'elle admet une espérance finie ($\mathbb{E}[|X_i|] < \infty$).

L'**espérance mathématique** du vecteur aléatoire X, notée $\mathbb{E}[X]$ (ou parfois μ), est le vecteur de $\mathbb{R}^p$ formé par les espérances de chacune de ses composantes :

$$\mathbb{E}[X] = \begin{pmatrix} \mathbb{E}[X_1] \\ \mathbb{E}[X_2] \\ \vdots \\ \mathbb{E}[X_p] \end{pmatrix}$$

Sous forme d'intégrale sur l'espace probabilisé Ω (lorsque la densité de probabilité conjointe f_X existe), cela s'écrit :

$$\mathbb{E}[X] = \int_{\mathbb{R}^p} x \, f_X(x) \, dx$$

Théorèmes: Propriétés fondamentales de l'espérance vectorielle

L'espérance d'un vecteur aléatoire vérifie les propriétés fondamentales suivantes :

1. **Linéarité :** Pour tous vecteurs aléatoires X et Y de même dimension et toutes matrices (ou constantes) déterministes compatibles A et B :

$$\mathbb{E}[AX + BY] = A\mathbb{E}[X] + B\mathbb{E}[Y]$$

2. **Extraction du déterminisme :** Toute matrice ou constante déterministe M sort de l'opérateur espérance :

$$\mathbb{E}[MX] = M\mathbb{E}[X]$$

Il s'agit en fait des mêmes propriétés déjà connues en dimension 1.

Démonstration:

1. Démonstration de la linéarité

Preuve par les composantes :

Soient $X = \begin{pmatrix} X_1 \\ \vdots \\ X_p \end{pmatrix}$ et $Y = \begin{pmatrix} Y_1 \\ \vdots \\ Y_p \end{pmatrix}$. Le vecteur somme pondérée $Z = aX + bY$ est un vecteur dont la i-ième composante, notée Z_i, s'écrit par définition de l'addition et de la multiplication externe des vecteurs :

$$Z_i = aX_i + bY_i$$

Par définition de l'espérance d'un vecteur, $\mathbb{E}[Z]$ est le vecteur dont chaque ligne est l'espérance de la composante Z_i :

$$\mathbb{E}[Z] = \begin{pmatrix} \mathbb{E}[Z_1] \\ \vdots \\ \mathbb{E}[Z_p] \end{pmatrix} = \begin{pmatrix} \mathbb{E}[aX_1 + bY_1] \\ \vdots \\ \mathbb{E}[aX_p + bY_p] \end{pmatrix}$$

Pour chaque composante i, X_i et Y_i sont des variables aléatoires scalaires. En appliquant la **linéarité de l'espérance en dimension 1** :

$$\mathbb{E}[aX_i + bY_i] = a\mathbb{E}[X_i] + b\mathbb{E}[Y_i]$$

En remplaçant dans le vecteur et en utilisant les règles d'addition et de multiplication par un scalaire des matrices colonnes :

$$\mathbb{E}[aX + bY] = \begin{pmatrix} a\mathbb{E}[X_1] + b\mathbb{E}[Y_1] \\ \vdots \\ a\mathbb{E}[X_p] + b\mathbb{E}[Y_p] \end{pmatrix} = a \begin{pmatrix} \mathbb{E}[X_1] \\ \vdots \\ \mathbb{E}[X_p] \end{pmatrix} + b \begin{pmatrix} \mathbb{E}[Y_1] \\ \vdots \\ \mathbb{E}[Y_p] \end{pmatrix} = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

La linéarité est démontrée. ■

---

2. Démonstration de l'extraction d'une matrice déterministe

Propriété : Soit M une matrice déterministe de dimension $m \times p$ et X un vecteur aléatoire de dimension $p \times 1$:

$$\mathbb{E}[MX] = M\mathbb{E}[X]$$

Preuve par la définition du produit matriciel : Soit $Y = MX$ le vecteur résultant de dimension $m \times 1$. Par la définition formelle du produit d'une matrice par un vecteur, chaque composante Y_i (pour $i \in \{1, \dots, m\}$) est une somme pondérée des composantes de X :

$$Y_i = \sum_{k=1}^{p} M_{ik} X_k$$

(Note : les coefficients M_{ik} sont des nombres réels fixes, connus et déterministes).

Appliquons la définition de l'espérance à la i-ième composante du vecteur Y :

$$\mathbb{E}[Y_i] = \mathbb{E}\left[\sum_{k=1}^{p} M_{ik} X_k\right]$$

Puisque l'espérance d'une somme finie est la somme des espérances, et que les constantes déterministes M_{ik} peuvent être sorties de l'espérance (propriété fondamentale de l'espérance scalaire) :

$$\mathbb{E}[Y_i] = \sum_{k=1}^{p} M_{ik} \mathbb{E}[X_k]$$

Reconnaissons maintenant cette expression : $\sum_{k=1}^{p} M_{ik}\mathbb{E}[X_k]$ est exactement la formule du i-ième coefficient du produit matriciel entre la matrice M et le vecteur colonne $\mathbb{E}[X] = \begin{pmatrix} \mathbb{E}[X_1] \\ \vdots \\ \mathbb{E}[X_p] \end{pmatrix}$.

Par conséquent, sous forme vectorielle :

$$\mathbb{E}[MX] = M\mathbb{E}[X]$$

L'extraction de la matrice déterministe est démontrée. ■

Définition : Matrice de variance d'un vecteur aléatoire

Soit X un vecteur aléatoire de dimension p et de vecteur espérance $\mu = \mathbb{E}[X]$. La **matrice de variance** du vecteur aléatoire X (notée $\mathrm{Var}(X)$ ou Σ) est la matrice carrée d'ordre p définie par l'espérance du produit externe du vecteur centré :

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mu)(X - \mu)^T\right]$$

Note : Le carré d'un vecteur n'a pas réellement le même sens algébriquement que le carré d'un scalaire. Le produit extérieur permet de mimer le comportement du carré. C'est pour cela par exemple que dans la décomposition de Cholesky, on retrouve ce produit extérieur. Le produit extérieur mimant donc l'effet du carré, la décomposition de Cholesky mime de ce fait la racine carrée, si l'on en suit cette logique.

Théorèmes : Propriétés fondamentales de la matrice de variance

La matrice de variance d'un vecteur aléatoire vérifie les propriétés fondamentales suivantes :

1. **Symétrie :** La matrice de variance est nécessairement une matrice symétrique :

$$\mathrm{Var}(X)^T = \mathrm{Var}(X)$$

2. **Positivité (Semi-définie positive) :** Pour tout vecteur déterministe $v \in \mathbb{R}^p$, la forme quadratique associée représente une variance réelle et est donc toujours positive ou nulle :

$$v^T \mathrm{Var}(X) v \geq 0$$

(La matrice est ainsi symétrique semi-définie positive, et définie positive si les composantes de X sont linéairement indépendantes).

3. **Transformation linéaire :** Pour toute matrice déterministe compatible A et tout vecteur constant b :

$$\mathrm{Var}(AX + b) = A \, \mathrm{Var}(X) \, A^T$$

Il s'agit en fait d'une extension matricielle directe des propriétés de la variance déjà connues en dimension 1, où le vecteur constant b disparaît par translation.

Démonstrations

Soit X un vecteur aléatoire de dimension p, de vecteur espérance $\mu = \mathbb{E}[X]$, et soit $\Sigma = \mathrm{Var}(X) = \mathbb{E}\left[(X - \mu)(X - \mu)^T\right]$.

1. Démonstration de la Symétrie ($\Sigma^T = \Sigma$)

Objectif : Montrer que la matrice de variance est égale à sa propre transposée.

- **Preuve :** Partons de la définition de Σ et appliquons la transposition à la matrice entière :

$$\Sigma^T = \left(\mathbb{E}\left[(X - \mu)(X - \mu)^T\right]\right)^T$$

Puisque l'opérateur espérance est une intégrale (ou une somme) agissant terme à terme, la transposition commute avec l'espérance :

$$\Sigma^T = \mathbb{E}\left[\left((X - \mu)(X - \mu)^T\right)^T\right]$$

Rappelons la règle de transposition d'un produit de matrices : $(AB)^T = B^T A^T$. En posant $A = (X - \mu)$ (vecteur colonne de dimension $p \times 1$) et $B = (X - \mu)^T$ (vecteur ligne de dimension $1 \times p$) :

$$\left((X-\mu)(X-\mu)^T\right)^T = \left((X-\mu)^T\right)^T(X-\mu)^T = (X-\mu)(X-\mu)^T$$

En remplaçant dans l'espérance :

$$\Sigma^T = \mathbb{E}\left[(X-\mu)(X-\mu)^T\right] = \Sigma$$

La symétrie est démontrée. ■

2. Démonstration de la Positivité / Semi-définie positivité ($v^T\Sigma v \geq 0$)

Objectif : Montrer que pour tout vecteur déterministe $v \in \mathbb{R}^p$, la forme quadratique $v^T\Sigma v$ est toujours supérieure ou égale à zéro.

- **Preuve :** Soit v un vecteur colonne déterministe de dimension $p \times 1$. Multiplions la matrice Σ par v^T à gauche et v à droite :

$$v^T\Sigma v = v^T\,\mathbb{E}\left[(X-\mu)(X-\mu)^T\right]\,v$$

En utilisant la linéarité de l'espérance et le fait que le vecteur v (étant déterministe) peut entrer et sortir librement de l'espérance :

$$v^T\Sigma v = \mathbb{E}\left[v^T(X-\mu)(X-\mu)^Tv\right]$$

Regardons le terme interne $v^T(X-\mu)$. C'est le produit d'un vecteur ligne par un vecteur colonne, ce qui donne un **nombre réel (un scalaire)**. Puisque c'est un scalaire, il est égal à sa propre transposée :

$$\left(v^T(X-\mu)\right)^T = (X-\mu)^T(v^T)^T = (X-\mu)^Tv$$

Le produit à l'intérieur de l'espérance se réécrit donc ainsi :

$$v^T(X-\mu)(X-\mu)^Tv = \left(v^T(X-\mu)\right)\left((X-\mu)^Tv\right) = \left(v^T(X-\mu)\right)^2$$

Posons $Y = v^T(X-\mu)$. C'est une variable aléatoire réelle (scalaire). L'expression devient :

$$v^T\Sigma v = \mathbb{E}[Y^2]$$

Or, pour n'importe quelle variable aléatoire réelle Y, son carré est toujours positif ou nul ($Y^2 \geq 0$), et par conséquent, son espérance l'est également ($\mathbb{E}[Y^2] \geq 0$). Par conséquent :

$$v^T \Sigma v \geq 0$$

La semi-définie positivité est démontrée. ■

3. Démonstration de la Transformation Linéaire ($\mathrm{Var}(AX+b) = A\mathrm{Var}(X)A^T$)

Objectif : Déterminer la matrice de variance du nouveau vecteur aléatoire $Y = AX + b$, où A est une matrice déterministe compatible ($m \times p$) et b un vecteur constant ($m \times 1$).

- **Preuve :** Commençons par calculer l'espérance du nouveau vecteur Y en utilisant les propriétés de linéarité et d'extraction du déterminisme :

$$\mu_Y = \mathbb{E}[Y] = \mathbb{E}[AX + b] = A\mathbb{E}[X] + b = A\mu_X + b$$

Centrons ensuite le vecteur Y :

$$Y - \mu_Y = (AX + b) - (A\mu_X + b) = AX - A\mu_X = A(X - \mu_X)$$

Appliquons la définition de la matrice de variance au vecteur Y :

$$\mathrm{Var}(Y) = \mathbb{E}\left[(Y - \mu_Y)(Y - \mu_Y)^T\right]$$

$$\mathrm{Var}(Y) = \mathbb{E}\left[(A(X - \mu_X))(A(X - \mu_X))^T\right]$$

Utilisons à nouveau la règle de transposition $(CD)^T = D^T C^T$ avec $C = A$ et $D = (X - \mu_X)$:

$$(A(X - \mu_X))^T = (X - \mu_X)^T A^T$$

L'expression du produit externe devient :

$$\mathrm{Var}(Y) = \mathbb{E}\left[A(X - \mu_X)(X - \mu_X)^T A^T\right]$$

Injectons cela dans l'espérance. Puisque A et A^T sont des matrices déterministes, elles se comportent comme des constantes vis-à-vis de l'espérance et peuvent être sorties de l'opérateur :

$$\mathrm{Var}(Y) = A\,\mathbb{E}\left[(X - \mu_X)(X - \mu_X)^T\right]\,A^T$$

Puisque le terme au milieu n'est autre que $\mathrm{Var}(X)$, on obtient :

$$\mathrm{Var}(AX + b) = A\,\mathrm{Var}(X)\,A^T$$

La formule de transformation linéaire est démontrée. ■

Théorème : Formule de König-Huygens matricielle

Soit X un vecteur aléatoire de dimension p admettant un vecteur espérance $\mu = \mathbb{E}[X]$ et une matrice de variance $\mathrm{Var}(X)$.

La matrice de variance de X peut s'exprimer par la différence entre l'espérance du produit externe brut et le produit externe du vecteur espérance :

$$\mathrm{Var}(X) = \mathbb{E}[XX^T] - \mu\mu^T$$

Démonstration

Objectif : Démontrer que la définition centrée de la matrice de variance est équivalente à la différence des moments d'ordre 2.

- **Preuve :** Partons de la définition théorique de la matrice de variance sous sa forme compacte :

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mu)(X - \mu)^T\right]$$

Étape 1 : Développement du produit interne Développons le produit des deux vecteurs à l'intérieur de l'espérance :

$$(X - \mu)(X - \mu)^T = (X - \mu)(X^T - \mu^T)$$
$$= XX^T - X\mu^T - \mu X^T + \mu\mu^T$$

Étape 2 : Application de la linéarité de l'espérance En appliquant l'opérateur espérance $\mathbb{E}[\cdot]$ terme à terme et en utilisant sa linéarité :

$$\mathrm{Var}(X) = \mathbb{E}\left[XX^T\right] - \mathbb{E}\left[X\mu^T\right] - \mathbb{E}\left[\mu X^T\right] + \mathbb{E}\left[\mu\mu^T\right]$$

Étape 3 : Simplification des termes constants Puisque le vecteur espérance $\mu = \mathbb{E}[X]$ est un vecteur déterministe (une constante par rapport à l'opérateur espérance), analysons chacun des termes de droite :

1. $\mathbb{E}[XX^T]$: C'est la matrice des moments bruts d'ordre 2.
2. $\mathbb{E}[X\mu^T]$: En sortant la constante déterministe μ^T à droite :

$$\mathbb{E}[X\mu^T] = \mathbb{E}[X]\mu^T = \mu\mu^T$$

3. $\mathbb{E}[\mu X^T]$: En sortant la constante déterministe μ à gauche :

$$\mathbb{E}[\mu X^T] = \mu\mathbb{E}[X]^T = \mu\mu^T$$

4. $\mathbb{E}[\mu\mu^T]$: L'espérance d'une matrice constante est la matrice elle-même :

$$\mathbb{E}[\mu\mu^T] = \mu\mu^T$$

Étape 4 : Assemblage final En remplaçant ces expressions dans l'équation développée, il vient :

$$\mathrm{Var}(X) = \mathbb{E}[XX^T] - \mu\mu^T - \mu\mu^T + \mu\mu^T$$

$$\mathrm{Var}(X) = \mathbb{E}[XX^T] - \mu\mu^T$$

La formule de König-Huygens matricielle est ainsi démontrée. ■

Définition : Indépendance de deux vecteurs aléatoires

Soient X un vecteur aléatoire de dimension p et Y un vecteur aléatoire de dimension q définis sur le même espace probabilisé $(\Omega, \mathcal{F}, \mathbb{P})$.

On dit que les vecteurs X et Y sont **indépendants** si, pour tous boréliens $A \subset \mathbb{R}^p$ et $B \subset \mathbb{R}^q$, les événements $\{X \in A\}$ et $\{Y \in B\}$ sont indépendants, c'est-à-dire :

$$\mathbb{P}(X \in A \text{ et } Y \in B) = \mathbb{P}(X \in A) \times \mathbb{P}(Y \in B)$$

- **Caractérisation par les densités (si elles existent) :** X et Y sont indépendants si et seulement si leur densité de probabilité conjointe $f_{X,Y}(x, y)$ est le produit de leurs densités marginales $f_X(x)$ et $f_Y(y)$:

$$f_{X,Y}(x, y) = f_X(x)\, f_Y(y)$$

Théorème : Découplage de l'espérance pour des vecteurs indépendants

Soient X un vecteur aléatoire de dimension p et Y un vecteur aléatoire de dimension q définis sur le même espace probabilisé. Si X et Y sont **indépendants**, alors pour toutes fonctions mesurables $g : \mathbb{R}^p \to \mathbb{R}$ et $h : \mathbb{R}^q \to \mathbb{R}$ telles que les espérances existent et sont finies, l'espérance de leur produit se factorise :

$$\mathbb{E}[g(X)\, h(Y)] = \mathbb{E}[g(X)]\, \mathbb{E}[h(Y)]$$

Démonstration

Objectif : Démontrer que l'espérance du produit de deux fonctions de vecteurs indépendants est égale au produit de leurs espérances respectives.

- **Preuve (dans le cas où les densités de probabilité conjointe et marginales existent) :**

Par définition de l'espérance d'une fonction de variables aléatoires multidimensionnelles, l'espérance de $g(X)h(Y)$ s'exprime via l'intégrale multiple sur l'espace produit $\mathbb{R}^p \times \mathbb{R}^q$ en utilisant la densité de probabilité conjointe $f_{X,Y}(x, y)$:

$$\mathbb{E}[g(X)\, h(Y)] = \int_{\mathbb{R}^p} \int_{\mathbb{R}^q} g(x)\, h(y)\, f_{X,Y}(x, y)\, dy\, dx$$

Par définition de l'**indépendance** de deux vecteurs aléatoires, la densité conjointe est le produit des densités marginales $f_X(x)$ et $f_Y(y)$:

$$f_{X,Y}(x, y) = f_X(x)\, f_Y(y)$$

Injectons cette relation dans l'intégrale double :

$$\mathbb{E}[g(X)\,h(Y)] = \int_{\mathbb{R}^p}\int_{\mathbb{R}^q} g(x)\,h(y)\,f_X(x)\,f_Y(y)\,dy\,dx$$

Réorganisons les termes de l'intégrande en regroupant tout ce qui dépend de y d'un côté et tout ce qui dépend de x de l'autre :

$$\mathbb{E}[g(X)\,h(Y)] = \int_{\mathbb{R}^p}\left(\int_{\mathbb{R}^q} h(y)\,f_Y(y)\,dy\right) g(x)\,f_X(x)\,dx$$

Observons l'intégrale intérieure entre parenthèses : $\int_{\mathbb{R}^q} h(y)\,f_Y(y)\,dy$. Cette quantité ne dépend absolument pas de la variable d'intégration extérieure x. C'est un **nombre réel fixe (une constante)**, qui correspond précisément par définition à l'espérance $\mathbb{E}[h(Y)]$:

$$\int_{\mathbb{R}^q} h(y)\,f_Y(y)\,dy = \mathbb{E}[h(Y)]$$

Puisque c'est une constante par rapport à la variable x, nous pouvons la sortir de l'intégrale extérieure :

$$\mathbb{E}[g(X)\,h(Y)] = \mathbb{E}[h(Y)]\int_{\mathbb{R}^p} g(x)\,f_X(x)\,dx$$

L'intégrale restante, $\int_{\mathbb{R}^p} g(x)\,f_X(x)\,dx$, n'est autre que la définition de l'espérance $\mathbb{E}[g(X)]$:

$$\int_{\mathbb{R}^p} g(x)\,f_X(x)\,dx = \mathbb{E}[g(X)]$$

En remplaçant, on obtient l'égalité finale :

$$\mathbb{E}[g(X)\,h(Y)] = \mathbb{E}[g(X)]\,\mathbb{E}[h(Y)]$$

Le théorème de découplage de l'espérance est ainsi démontré. ■

Jusqu'à présent, nous avons posé l'opérateur $\mathrm{Var}(X)$, qui désigne la matrice globale Σ. Regardons en détail sa structure interne (ce qui se cache dans ses lignes et ses colonnes) et introduisons son extension naturelle entre deux vecteurs différents : **la matrice de**

covariance croisée.

Définition: Structure détaillée de la matrice de covariance d'un vecteur**

Soit X un vecteur aléatoire de dimension p :

$$X = \begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{pmatrix}$$

Sa matrice de covariance, notée $\mathrm{Cov}(X)$ (ou Σ), est une matrice carrée d'ordre p dont chaque élément positionné à la ligne i et à la colonne j est défini par la covariance entre la i-ième et la j-ième composante de X :

$$[\mathrm{Cov}(X)]_{ij} = \mathrm{Cov}(X_i, X_j) = \mathbb{E}\left[(X_i - \mu_i)(X_j - \mu_j)\right]$$

Sous forme tabulaire explicite, cela donne :

$$\mathrm{Cov}(X) = \begin{pmatrix} \mathrm{Cov}(X_1, X_1) & \mathrm{Cov}(X_1, X_2) & \cdots & \mathrm{Cov}(X_1, X_p) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Cov}(X_2, X_2) & \cdots & \mathrm{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(X_p, X_1) & \mathrm{Cov}(X_p, X_2) & \cdots & \mathrm{Cov}(X_p, X_p) \end{pmatrix}$$

- **Sur la diagonale principale ($i = j$) :** On retrouve les variances de chaque composante, puisque $\mathrm{Cov}(X_i, X_i) = \mathrm{Var}(X_i)$.
- **Hors de la diagonale ($i \neq j$) :** On retrouve les covariances croisées entre les différentes variables du système. Comme $\mathrm{Cov}(X_i, X_j) = \mathrm{Cov}(X_j, X_i)$, la matrice est par construction **symétrique**.

Définition : La matrice de covariance croisée entre deux vecteurs

- **Définition :** Soient X un vecteur aléatoire de dimension p (d'espérance μ_X) et Y un vecteur aléatoire de dimension q (d'espérance μ_Y) définis sur le même espace probabilisé. La **matrice de covariance croisée** entre X et Y, notée $\mathrm{Cov}(X, Y)$, est la matrice de dimension $p \times q$ définie par :

$$\mathrm{Cov}(X, Y) = \mathbb{E}\left[(X - \mu_X)(Y - \mu_Y)^T\right]$$

- **Remarque de dimension :** Contrairement à $\mathrm{Var}(X)$ qui est toujours carrée et symétrique, la matrice de covariance croisée $\mathrm{Cov}(X, Y)$ n'est ni carrée (sauf si $p = q$) ni symétrique. Son terme général de ligne i et de colonne j est $\mathrm{Cov}(X_i, Y_j)$.

Théorèmes: Propriétés fondamentales de la covariance croisée**

La matrice de covariance croisée vérifie des règles de calcul algébrique strictes, très utiles pour les changements de variables en finance :

1. **Relation de transposition :** Inverser l'ordre des vecteurs revient à transposer la matrice de covariance croisée :

$$\mathrm{Cov}(Y, X) = \mathrm{Cov}(X, Y)^T$$

2. **Bilinéarité (Compatibilité avec les transformations linéaires) :** Si l'on applique des transformations linéaires $AX + b$ et $CY + d$ (avec des dimensions compatibles) :

$$\mathrm{Cov}(AX + b,\, CY + d) = A\,\mathrm{Cov}(X, Y)\,C^T$$

(Note : les vecteurs constants b et d disparaissent également par translation, car la covariance ne dépend que des fluctuations autour des espérances).

3. **Cas des vecteurs indépendants :** Si les vecteurs X et Y sont **indépendants**, alors leur matrice de covariance croisée est entièrement nulle :

$$\mathrm{Cov}(X, Y) = 0_{p \times q}$$

Démonstrations : Propriétés de la covariance croisée

Soient X un vecteur aléatoire de dimension p (d'espérance μ_X) et Y un vecteur aléatoire de dimension q (d'espérance μ_Y). La matrice de covariance croisée est définie par :

$$\mathrm{Cov}(X, Y) = \mathbb{E}\left[(X - \mu_X)(Y - \mu_Y)^T\right]$$

---

1. Démonstration de la Relation de Transposition ($\mathrm{Cov}(Y, X) = \mathrm{Cov}(X, Y)^T$)

Objectif : Montrer que la matrice de covariance croisée inversée est égale à la transposée de la matrice initiale.

- **Preuve :** Partons de la définition de la covariance croisée en inversant les rôles de X et Y :

$$\mathrm{Cov}(Y, X) = \mathbb{E}\left[(Y - \mu_Y)(X - \mu_X)^T\right]$$

Appliquons l'opérateur de transposition à cette matrice entière. Puisque la transposition commute avec l'espérance :

$$\mathrm{Cov}(Y, X) = \mathbb{E}\left[\left((Y - \mu_Y)(X - \mu_X)^T\right)^T\right]$$

Rappelons la règle de transposition d'un produit de matrices : $(AB)^T = B^T A^T$. En posant $A = (Y - \mu_Y)$ (vecteur de dimension $q \times 1$) et $B = (X - \mu_X)^T$ (vecteur de dimension $1 \times p$) :

$$\left((Y - \mu_Y)(X - \mu_X)^T\right)^T = \left((X - \mu_X)^T\right)^T (Y - \mu_Y)^T = (X - \mu_X)(Y - \mu_Y)^T$$

En remplaçant cette expression à l'intérieur de l'espérance, on reconnaît la définition de $\mathrm{Cov}(X, Y)$:

$$\mathrm{Cov}(Y, X) = \mathbb{E}\left[(X - \mu_X)(Y - \mu_Y)^T\right]^T = \mathrm{Cov}(X, Y)^T$$

La relation de transposition est démontrée. ■

---

2. Démonstration de la Bilinéarité / Transformation Linéaire ($\mathrm{Cov}(AX + b,\, CY + d) = A\,\mathrm{Cov}(X, Y)\,C^T$)

Objectif : Déterminer la matrice de covariance croisée entre deux nouveaux vecteurs transformés $U = AX + b$ et $V = CY + d$, où A est de dimension $m \times p$, C de dimension $k \times q$, et b, d sont des vecteurs constants de dimensions respectives $m \times 1$ et $k \times 1$.

- **Preuve :** Commençons par calculer les espérances de ces nouveaux vecteurs par linéarité :

$$\mu_U = \mathbb{E}[AX + b] = A\mathbb{E}[X] + b = A\mu_X + b$$

$$\mu_V = \mathbb{E}[CY + d] = C\mathbb{E}[Y] + d = C\mu_Y + d$$

Centrons ensuite les deux vecteurs :

$$U - \mu_U = (AX + b) - (A\mu_X + b) = AX - A\mu_X = A(X - \mu_X)$$

$$V - \mu_V = (CY + d) - (C\mu_Y + d) = CY - C\mu_Y = C(Y - \mu_Y)$$

Appliquons la définition de la matrice de covariance croisée au couple (U, V) :

$$\mathrm{Cov}(U, V) = \mathbb{E}\left[(U - \mu_U)(V - \mu_V)^T\right]$$

$$\mathrm{Cov}(U, V) = \mathbb{E}\left[(A(X - \mu_X))(C(Y - \mu_Y))^T\right]$$

Utilisons la règle de transposition sur le second terme ($(CD)^T = D^T C^T$) :

$$(C(Y - \mu_Y))^T = (Y - \mu_Y)^T C^T$$

L'expression à l'intérieur de l'espérance devient :

$$(U - \mu_U)(V - \mu_V)^T = A\,(X - \mu_X)(Y - \mu_Y)^T\,C^T$$

Puisque A et C^T sont des matrices entièrement déterministes, elles se comportent comme des constantes vis-à-vis de l'espérance et peuvent être sorties de l'opérateur :

$$\mathrm{Cov}(U, V) = A\,\mathbb{E}\left[(X - \mu_X)(Y - \mu_Y)^T\right]\,C^T$$

Puisque le terme central n'est autre que $\mathrm{Cov}(X, Y)$, on obtient l'égalité finale :

$$\mathrm{Cov}(AX + b,\, CY + d) = A\,\mathrm{Cov}(X, Y)\,C^T$$

La formule de bilinéarité est démontrée. ■

---

3. Démonstration du Cas des Vecteurs Indépendants ($\mathrm{Cov}(X, Y) = 0_{p \times q}$)

Objectif : Montrer que si les vecteurs X et Y sont indépendants, leur matrice de covariance croisée est la matrice nulle.

- **Preuve :** Analysons le terme général de la matrice de covariance croisée, qui correspond à la covariance entre la i-ième composante de X et la j-ième composante de Y :

$$[\mathrm{Cov}(X, Y)]_{ij} = \mathrm{Cov}(X_i, Y_j) = \mathbb{E}\left[(X_i - \mu_{X_i})(Y_j - \mu_{Y_j})\right]$$

Si les vecteurs X et Y sont **indépendants**, alors par définition, n'importe quelle fonction mesurable de X et n'importe quelle fonction mesurable de Y sont des variables aléatoires scalaires indépendantes. En particulier, les variables $(X_i - \mu_{X_i})$ et $(Y_j - \mu_{Y_j})$ le sont.

En appliquant le théorème de **découplage de l'espérance** pour des variables indépendantes :

$$\mathbb{E}\left[(X_i - \mu_{X_i})(Y_j - \mu_{Y_j})\right] = \mathbb{E}[X_i - \mu_{X_i}] \times \mathbb{E}[Y_j - \mu_{Y_j}]$$

Or, par linéarité de l'espérance, l'espérance d'une variable centrée est nulle :

$$\mathbb{E}[X_i - \mu_{X_i}] = \mathbb{E}[X_i] - \mu_{X_i} = \mu_{X_i} - \mu_{X_i} = 0$$

De même, $\mathbb{E}[Y_j - \mu_{Y_j}] = 0$.

Par conséquent, chaque coefficient de la matrice de covariance croisée est égal à zéro :

$$[\mathrm{Cov}(X, Y)]_{ij} = 0 \times 0 = 0$$

Sous forme matricielle globale, tous les éléments étant nuls, la matrice est la matrice nulle de dimension $p \times q$:

$$\mathrm{Cov}(X, Y) = 0_{p \times q}$$

La nullité de la covariance croisée en cas d'indépendance est démontrée. ■

Déterminant de la matrice de covariance

La matrice étant symétrique, le théorème spéctral nous assure que la matrice de covariance est diagonalisable et on écrira ses valeurs propres :

$$(\lambda_i)_{i\in[1,n]}$$

On notera $diag(\lambda_i)_{i\in[1,n]}$ la matrice diagonale obtenue après diagonalisation de la matrice de covariance.

Certaines de ces valeurs propres sont potentiellements confondues et quitte à les réordonner, on supposera qu'elles sont ordonnées dans l'ordre décroissant.

On en déduit donc que:

$$Var_{totale}(X) = \sum_{k=1}^{n} Var(X_k) = Tr(\Sigma_X) = Tr(diag(\lambda_i)_{i\in[1,n]}) = \sum_{k=1}^{n} \lambda_k$$

Le déterminant de la matrice de covariance est donc:

$$det(\Sigma_X) = \prod_{k=1}^{n} \lambda_k$$

Si la matrice de covariance est non-inversible alors $\exists j \in [\![1,n]\!], \lambda_j = 0$ donc $det(\Sigma_X) = 0$.

Cela implique donc aussi la colinéarité de 1 ou plusieurs vecteurs aléatoires X_k pour $k \in [\![1,n]\!]$, puisque la matrice de covariance n'est alors que définie semi-positive.

Supposons maintenant la non-corrélation 2 à 2 des vecteurs $(X_k)_{k\in[1,n]}$ de variance toutes non-nulles. La matrice Σ_X serait alors:

$$\Sigma_X = diag(Var(X_k)_{k\in[1,n]})$$

Le déterminant serait alors:

$$det(\Sigma_X) = \prod_{k=1}^{n} Var(X_k)$$

C'est donc un pavé droit formé par les intervalles $\pm Std(X_K)$, si l'on en revient à l'échelle du jeu de données formés par les vecteurs $(X_k)_{k\in[1,n]}$.

Toutefois, la matrice de Covariance peut s'écrire à l'aide de la matrice de corrélation R_X de manière ci-suit:

$$\Sigma_X = diag(Std(X_k)_{k\in[1,n]}) \times R_X \times diag(Std(X_k)_{k\in[1,n]})$$

En posant $D = diag(Std(X_k)_{k\in[1,n]})$ pour simplifier les notiations, on en déduit:

$$Det(\Sigma_X) = Det(D) \times Det(R_X) \times Det(D) = Det(D)^2 \times Det(R_X) = Det(D^2) \times Det(R_X)$$

Or, $Det(D)^2$ est lui même notre volume du pavé droit obtenu en situation de totale indépendance. Ce volume est alors soumis à une déformation liée à l'aire obtenue par les corrélations des variables entre elles.

Il reste toutefois l'obligation de quantifier une borne supérieure pour le déterminant de R_x.La formule de Hadamard nous donnera :

$$\det(R_X) \leq \prod_{i=1}^{n} (R_X)_{ii}$$

Puisque les éléments diagonaux de la matrice de corrélation R_X valent tous 1 (chaque variable étant parfaitement corrélée avec elle-même), ce produit se simplifie immédiatement :

$$\det(R_X) \leq \prod_{i=1}^{n} 1 = 1$$

Ainsi, le déterminant de la matrice de corrélation est strictement borné par 1. Par conséquent, le volume réel de notre déterminant est inférieur ou égal à celui du déterminant dans le cas d'une indépendance totales des $(X_k)_{k\in[1,n]}$.

Il reste à savoir qu'est-ce que mesure réellement comme volume le déterminant de notre matrice de covariance dans ce cas aussi général.

Puisque la matrice de covariance Σ est symétrique et définie positive, le théorème spectral nous permet de la diagonaliser :

$$\Sigma = Q\Lambda Q^T$$

Donc :

$$\Sigma^{-1} = (Q\Lambda Q^T)^{-1} = (Q^T)^{-1}\Lambda^{-1}Q^{-1}$$

Puisque Q est une matrice orthogonale, ses propriétés fondamentales nous donnent $Q^{-1} = Q^T$ et $(Q^T)^{-1} = Q$. De plus, l'inverse de la matrice diagonale Λ est simplement constituée de l'inverse de ses éléments diagonaux, soit $diag\left(\frac{1}{\lambda_i}\right)$. L'expression de l'inverse devient alors :

$$\Sigma^{-1} = Q\, diag\left(\frac{1}{\lambda_i}\right) Q^T$$

Introduisons maintenant cette expression dans la forme quadratique générale qui définit la surface de dispersion du nuage de points :

$$x^T\Sigma^{-1}x = c^2$$

C'est-à-dire :

$$x^T\left(Q\, diag\left(\frac{1}{\lambda_i}\right) Q^T\right)x = c^2$$

Effectuons un changement de variable en posant un nouveau vecteur de coordonnées $y = Q^T x$ (ce qui équivaut géométriquement à une **rotation** du repère alignée sur les axes principaux de la matrice de covariance). Par transposition, on a $x = Qy$. L'équation devient :

$$(Qy)^T\left(Q\, diag\left(\frac{1}{\lambda_i}\right) Q^T\right)(Qy) = c^2$$

$$y^T Q^T Q\, diag\left(\frac{1}{\lambda_i}\right) Q^T Q\, y = c^2$$

Puisque Q est orthogonale, $Q^T Q = I$ (la matrice identité). L'expression se simplifie radicalement :

$$y^T diag\left(\frac{1}{\lambda_i}\right) y = c^2$$

Sous sa forme développée, cette équation s'écrit :

$$\sum_{k=1}^{n} \frac{y_k^2}{\lambda_k} = c^2 \quad \text{soit} \quad \frac{y_1^2}{(\sqrt{c^2\lambda_1})^2} + \frac{y_2^2}{(\sqrt{c^2\lambda_2})^2} + \cdots + \frac{y_n^2}{(\sqrt{c^2\lambda_n})^2} = 1$$

Conclusion géométrique : Pourquoi obtient-on un ellipsoïde et non un pavé droit ?

Cette dernière équation est l'expression canonique absolue d'un **ellipsoïde** orienté selon les axes principaux de l'espace, dont les demi-longueurs d'axes sont proportionnelles aux racines carrées des valeurs propres ($\sqrt{\lambda_k}$).

- **Le pavé droit** initial correspondait à une vision indépendante axe par axe ($\pm Std(X_k)$), formant une boîte rigide aux angles droits délimitée par les variances marginales.
- **L'ellipsoïde** émerge mathématiquement de la diagonalisation : la matrice de covariance (via ses vecteurs propres Q) pivote l'espace, et (via ses valeurs propres λ_k) ajuste l'étirement ou la compression le long de chaque nouvel axe.

Ainsi, le déterminant de la matrice de covariance, $\det(\Sigma_X) = \prod_{k=1}^{n} \lambda_k$, mesure précisément le **produit des carrés des demi-axes** de cet ellipsoïde (à une constante géométrique près liée à la dimension).

En reliant cela à notre décomposition $\det(\Sigma_X) = \det(D)^2 \times \det(R_X)$, nous confirmons que :

1. $\det(D)^2$ représente le volume de référence du pavé droit (sans corrélation).
2. $\det(R_X)$ (dont nous avons prouvé grâce à Hadamard que $\det(R_X) \leq 1$) agit comme un **facteur de contraction géométrique**.

La corrélation ne se contente pas de modifier une formule : elle "rabote et arrondit" les coins du pavé droit d'origine en inclinant l'espace, forçant le nuage de points à s'inscrire dans un volume d'ellipsoïde strictement inférieur ou égal au pavé initial.

In [188…

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# 1. Paramètres et génération des données
np.random.seed(42)
n_samples = 1000
mu = np.array([0, 0])
```

```
v1, v2 = 4.0, 1.0
sigma1 = np.sqrt(v1)       # 2.0
sigma2 = np.sqrt(v2)       # 1.0
rho = 0.5

cov12 = rho * sigma1 * sigma2
sigma_corr = np.array([[v1, cov12], [cov12, v2]])
R_X = np.array([[1.0, rho], [rho, 1.0]])
det_Rx = np.linalg.det(R_X)  # 0.75

# Données réelles et données normalisées (espace de R_X)
data_x = np.random.multivariate_normal(mu, sigma_corr, size=n_samples)
data_z = data_x @ np.linalg.inv(np.diag([sigma1, sigma2]))

# Grilles partagées pour une cohérence parfaite des contours
grid_range = np.linspace(-6, 6, 200)
xx, yy = np.meshgrid(grid_range, grid_range)
pos = np.dstack((xx, yy))
rv_rx = multivariate_normal(np.array([0, 0]), R_X)
rv_sigma = multivariate_normal(mu, sigma_corr)

# 2. Tracé à deux fenêtres avec des limites d'axes un peu plus larges pour le texte
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 7))
lim_min, lim_max = -5.5, 5.5  # Limites élargies pour intégrer proprement les étiquettes

# --- GRAPHIQUE 1 : L'espace de R_X (Espace normalisé Z) ---
ax1.scatter(data_z[:, 0], data_z[:, 1], color='black', alpha=0.15, s=20, label='Données normalisées $Z$')

# Carré indépendant normalisé de base ([-1, 1] x [-1, 1])
square_z = plt.Rectangle((-1, -1), 2, 2, fill=True, facecolor='dodgerblue', alpha=0.1,
                         edgecolor='dodgerblue', linestyle='--', linewidth=2, label='Carré de base ([-1, 1])')
ax1.add_patch(square_z)

# Surbrillance en rose des aires ajoutées par l'étirement des écarts-types
pink_left = plt.Rectangle((-sigma1, -sigma2), sigma1 - 1, 2*sigma2, fill=True, facecolor='hotpink', alpha=0.3, edge
pink_right = plt.Rectangle((1, -sigma2), sigma1 - 1, 2*sigma2, fill=True, facecolor='hotpink', alpha=0.3, edgecolor
ax1.add_patch(pink_left)
ax1.add_patch(pink_right)

# Contour global du pavé étiré en violet
rect_purple = plt.Rectangle((-sigma1, -sigma2), 2*sigma1, 2*sigma2, fill=False,
```

```
                            edgecolor='purple', linestyle='-', linewidth=2.5,
                            label='Pavé total étiré ($\pm\sigma_1, \pm\sigma_2$)')
ax1.add_patch(rect_purple)

# Ellipse de R_X pure
contour_rx = ax1.contour(xx, yy, rv_rx.pdf(pos), levels=[0.02], colors='crimson', linewidths=3)

# Étiquette fixe pour l'ellipse de gauche
ax1.text(0.8, 1.4, f'Ellipse de $R_X$\n($\\det = {det_Rx}$)', color='crimson',
         fontsize=9, fontweight='bold', bbox=dict(facecolor='white', alpha=0.7, edgecolor='none'))

# Lignes de repère
ax1.axvline(x=1, color='gray', linestyle=':', alpha=0.7)
ax1.axvline(x=-1, color='gray', linestyle=':', alpha=0.7)
ax1.axhline(y=1, color='gray', linestyle='-.', alpha=0.7)
ax1.axhline(y=-1, color='gray', linestyle='-.', alpha=0.7)

# Droite OLS dans l'espace normalisé
beta_1_z = rho
z_line = np.linspace(-5, 5, 100)
ax1.plot(z_line, beta_1_z * z_line, color='forestgreen', linestyle='-', linewidth=2.5, label=f'Droite OLS (pente =

ax1.set_title(f"1. Le monde de $R_X$ (Espace normalisé)\nL'extension rose montre l'effet d'échelle de la matrice $D
ax1.set_xlabel('$Z_1$')
ax1.set_ylabel('$Z_2$')
ax1.set_aspect('equal')
ax1.grid(True, linestyle='--', alpha=0.4)
ax1.legend(loc='upper right', fontsize=8)
ax1.set_xlim(lim_min, lim_max)
ax1.set_ylim(lim_min, lim_max)


# --- GRAPHIQUE 2 : L'espace Réel ($\Sigma = D R_X D$) ---
ax2.scatter(data_x[:, 0], data_x[:, 1], color='black', alpha=0.15, s=20, label='Données réelles $X$')

# Rectangle des variances (Pavé réel)
rect_x = plt.Rectangle((-sigma1, -sigma2), 2*sigma1, 2*sigma2, fill=True, facecolor='dodgerblue', alpha=0.1,
                       edgecolor='dodgerblue', linestyle='--', linewidth=2, label='Pavé des variances (Aire = 4.0)'
ax2.add_patch(rect_x)

# Ellipse réelle
```

```python
contour_sig = ax2.contour(xx, yy, rv_sigma.pdf(pos), levels=[0.02], colors='crimson', linewidths=3)

# Étiquette fixe pour l'ellipse de droite avec fond blanc semi-transparent
ax2.text(1.8, 1.1, f'Ellipse réelle $\\Sigma$\n(Aire = 3.0)', color='crimson',
         fontsize=9, fontweight='bold', bbox=dict(facecolor='white', alpha=0.7, edgecolor='none'))

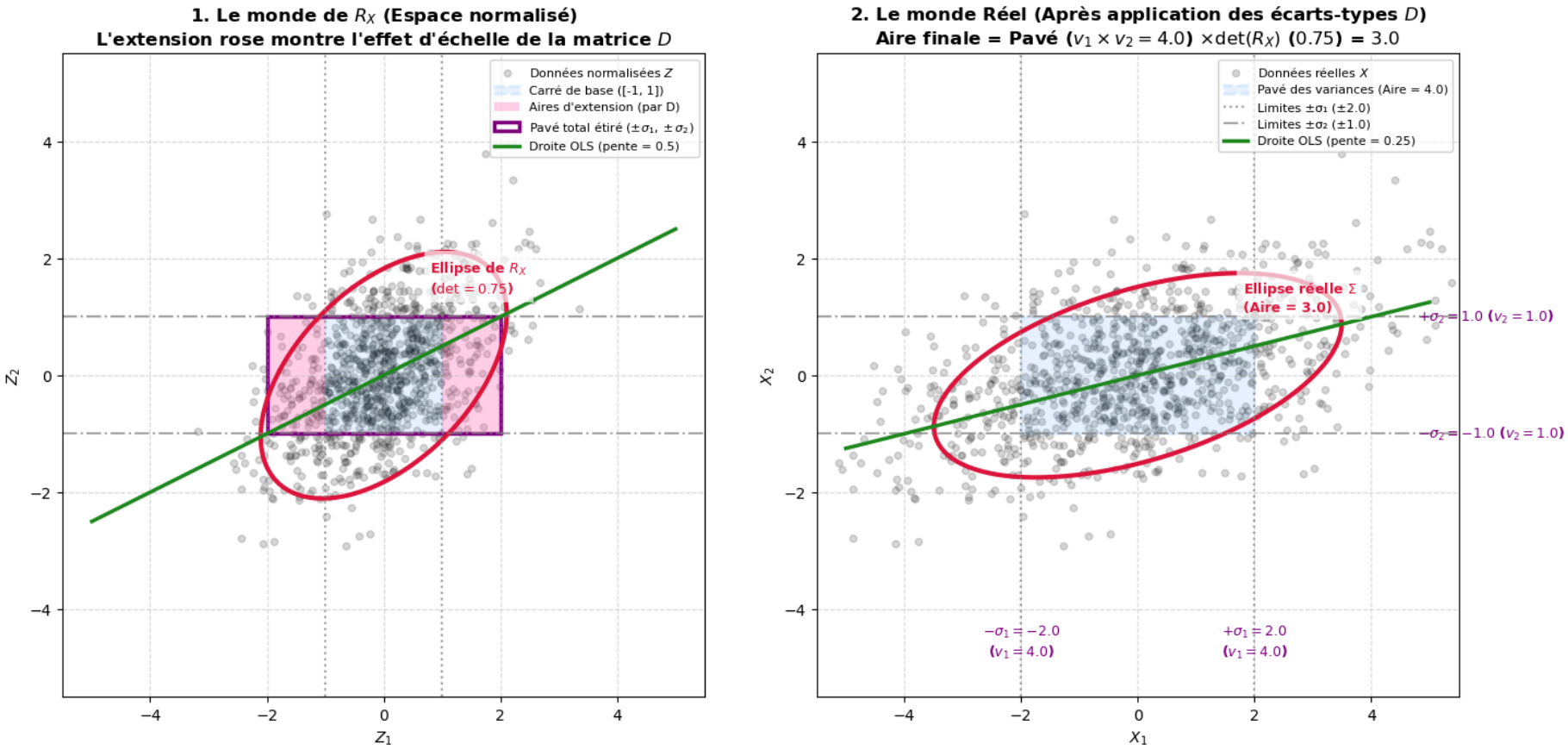
# Lignes de repère des écarts-types
ax2.axvline(x=sigma1, color='gray', linestyle=':', alpha=0.7)
ax2.axvline(x=-sigma1, color='gray', linestyle=':', alpha=0.7, label=f'Limites ±σ₁ (±{sigma1})')
ax2.axhline(y=sigma2, color='gray', linestyle='-.', alpha=0.7)
ax2.axhline(y=-sigma2, color='gray', linestyle='-.', alpha=0.7, label=f'Limites ±σ₂ (±{sigma2})')

# Ajout des indications de variances/écarts-types sur les axes de droite
ax2.text(sigma1, -4.8, f'$+σ_1 = {sigma1}$\n($v_1 = {v1}$)', color='purple', ha='center', fontsize=9, fontweight='b
ax2.text(-sigma1, -4.8, f'$-σ_1 = -{sigma1}$\n($v_1 = {v1}$)', color='purple', ha='center', fontsize=9, fontweight=
ax2.text(4.8, sigma2, f'$+σ_2 = {sigma2}$ ($v_2 = {v2}$)', color='purple', va='center', fontsize=9, fontweight='bol
ax2.text(4.8, -sigma2, f'$-σ_2 = -{sigma2}$ ($v_2 = {v2}$)', color='purple', va='center', fontsize=9, fontweight='b

# Droite OLS dans l'espace réel
beta_1_x = cov12 / v1
x_line = np.linspace(-5, 5, 100)
ax2.plot(x_line, beta_1_x * x_line, color='forestgreen', linestyle='-', linewidth=2.5, label=f'Droite OLS (pente =

ax2.set_title("2. Le monde Réel (Après application des écarts-types $D$)\nAire finale = Pavé ($v_1 \\times v_2 = 4.
ax2.set_xlabel('$X_1$')
ax2.set_ylabel('$X_2$')
ax2.set_aspect('equal')
ax2.grid(True, linestyle='--', alpha=0.4)
ax2.legend(loc='upper right', fontsize=8)
ax2.set_xlim(lim_min, lim_max)
ax2.set_ylim(lim_min, lim_max)

plt.tight_layout()
plt.show()
```



Définition : L'inverse de Moore-Penrose (Pseudo-inverse)

Soit A une matrice réelle de dimension $m \times n$ (qui peut être rectangulaire ou carrée non inversible). L'**inverse de Moore-Penrose**, noté A^+, est l'unique matrice de dimension $n \times m$ qui satisfait les quatre **conditions de Penrose** :

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$ (la matrice AA^+ est symétrique)
4. $(A^+A)^T = A^+A$ (la matrice A^+A est symétrique)

Cas particulier fondamental (Matrice symétrique semi-définie positive) :

Si A est une matrice symétrique semi-définie positive (comme une sous-matrice de variance Σ_{11}) de décomposition spectrale $A = P\Lambda P^T$, où $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ contient des valeurs propres ≥ 0, son inverse de Moore-Penrose s'obtient en inversant uniquement les valeurs propres strictement positives et en laissant les zéros intacts :

$$A^+ = P\Lambda^+ P^T \quad \text{avec} \quad \Lambda^+ = \text{diag}\left(\frac{1}{\lambda_1}, \dots, \frac{1}{\lambda_r}, 0, \dots, 0\right)$$

(où r est le rang de la matrice).

---

2. Définition : La régression linéaire multivariée par projection

Soient X_1 un vecteur explicatif de dimension k (d'espérance μ_1) et X_2 un vecteur à expliquer de dimension $p-k$ (d'espérance μ_2).

On cherche à modéliser X_2 comme une fonction linéaire de X_1 sous la forme :

$$\hat{X}_2 = AX_1 + b$$

Le problème de la **régression linéaire au sens des moindres carrés** consiste à trouver la matrice de coefficients A et le vecteur constant b qui minimisent l'espérance de la norme au carré de l'erreur résiduelle (l'erreur quadratique moyenne) :

$$\min_{A,b} \mathbb{E}\left[\|X_2 - (AX_1 + b)\|^2\right]$$

Résolution par les équations normales :

1. **Pour le terme constant b :** En dérivant par rapport à b et en annulant le gradient, on obtient immédiatement :
$$b = \mu_2 - A\mu_1$$
2. **Pour la matrice de régression A :** En injectant b et en minimisant par rapport à A, on obtient l'équation normale :
$$\text{Cov}(X_2, X_1) = A\,\text{Cov}(X_1) \implies \Sigma_{21} = A\,\Sigma_{11}$$

Si Σ_{11} est inversible, on isole A classiquement : $A = \Sigma_{21}\Sigma_{11}^{-1}$. **Mais si Σ_{11} est singulière (présence de multicolinéarité dans X_1), on utilise le pseudoinverse de Moore-Penrose :**

$$A = \Sigma_{21}\Sigma_{11}^{+}$$

Conséquence directe :

La meilleure approximation linéaire (la projection) de X_2 sachant X_1 s'écrit rigoureusement :

$$\hat{X}_2 = \mu_2 + \Sigma_{21}\Sigma_{11}^{+}(X_1 - \mu_1)$$

Définition : Covariance résiduelle (Matrice de covariance résiduelle généralisée)

La **covariance résiduelle** (ou matrice de covariance conditionnelle/du résidu) quantifie l'incertitude ou la variance qui subsiste sur un sous-vecteur aléatoire X_2 après avoir éliminé toute l'information linéaire apportée par un autre sous-vecteur X_1, et ce, même en présence de redondances (multicolinéarité) grâce à l'utilisation du pseudoinverse de Moore-Penrose.

1. Construction du vecteur résiduel par projection linéaire généralisée

Soit un vecteur aléatoire global partitionné en deux blocs :

$$X = \begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \quad \text{de vecteur espérance} \quad \mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}$$

Et dont la matrice de covariance globale Σ est découpée par blocs :

$$\Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$$

(avec $\Sigma_{21} = \Sigma_{12}^T$ par symétrie).

Supposons que l'on veuille estimer ou prédire linéairement le bloc X_2 en fonction du bloc X_1. La meilleure approximation linéaire de X_2 sachant X_1 (au sens des moindres carrés, généralisée via le pseudoinverse de Moore-Penrose Σ_{11}^{+} pour parer d'éventuelles singularités) est donnée par la projection linéaire :

$$\hat{X}_2 = \mu_2 + \Sigma_{21}\Sigma_{11}^{+}(X_1 - \mu_1)$$

Le **vecteur résiduel** R_2 (l'erreur de cette projection linéaire) est la part de X_2 qui n'est *pas* expliquée par X_1 :

$$R_2 = X_2 - \hat{X}_2 = (X_2 - \mu_2) - \Sigma_{21}\Sigma_{11}^{+}(X_1 - \mu_1)$$

---

2. Définition mathématique de la covariance résiduelle

La **covariance résiduelle** du bloc X_2 par rapport à X_1 est par définition la matrice de variance de ce vecteur d'erreur R_2 :

$$\mathrm{Cov}(R_2) = \mathbb{E}\left[(R_2 - \mathbb{E}[R_2])(R_2 - \mathbb{E}[R_2])^T\right]$$

Puisque l'espérance mathématique de l'erreur de projection est nulle ($\mathbb{E}[R_2] = 0$), cela se simplifie en :

$$\mathrm{Cov}(R_2) = \mathbb{E}\left[R_2 R_2^T\right]$$

---

3. Lien fondamental avec le Complément de Schur généralisé

Si l'on développe le calcul de $\mathbb{E}[R_2 R_2^T]$ en remplaçant R_2 par son expression, les termes croisés s'annulent par orthogonalité des résidus de projection, et il reste exactement :

$$\mathrm{Cov}(R_2) = \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{+}\Sigma_{12}$$

On reconnaît de manière immédiate le **complément de Schur généralisé** S :

$$\mathrm{Cov}(R_2) = S$$

Interprétation :

- **La covariance résiduelle (S) est la variance de l'erreur de prédiction.**
- Grâce à l'utilisation de Σ_{11}^{+}, cette définition reste mathématiquement valide même si le bloc explicatif X_1 contient des redondances (variables colinéaires). Si une variable est parfaitement redondante, sa projection absorbe l'excès et la variance résiduelle s'annule proprement dans cette direction.
- En univers gaussien, elle correspond exactement à la **matrice de covariance conditionnelle** $\mathrm{Cov}(X_2 \mid X_1)$. C'est l'incertitude "nette" qui subsiste sur X_2 lorsqu'on a extrait l'effet de X_1.

Propriété : Du complément de Schur généralisé au complément standard (Cas défini positif)

Lorsque la sous-matrice explicative Σ_{11} est **symétrique définie positive** ($\Sigma_{11} \succ 0$), le cadre général (utilisant le pseudoinverse de Moore-Penrose) se simplifie naturellement pour retrouver l'expression classique du complément de Schur.

1. Propriété fondamentale de l'inverse pour les matrices pleines

Si Σ_{11} est une matrice carrée d'ordre k symétrique définie positive, elle est de **plein rang** ($r = k$). Toutes ses valeurs propres $\lambda_1, \lambda_2, \dots, \lambda_k$ sont strictement positives.

Par conséquent, dans sa décomposition spectrale $\Sigma_{11} = P\Lambda P^T$, la matrice diagonale des inverses Λ^+ ne contient aucun zéro :

$$\Lambda^+ = \mathrm{diag}\left(\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \dots, \frac{1}{\lambda_k}\right) = \Lambda^{-1}$$

Il en découle directement que le pseudoinverse de Moore-Penrose coïncide exactement avec l'inverse classique :

$$\Sigma_{11}^{+} = P\Lambda^{+}P^T = P\Lambda^{-1}P^T = \Sigma_{11}^{-1}$$

---

2. Transition vers le complément de Schur standard

En injectant cette égalité ($\Sigma_{11}^{+} = \Sigma_{11}^{-1}$) dans la formule générale de la covariance résiduelle (le complément de Schur généralisé) :

$$\Sigma_{22} - \Sigma_{21}\Sigma_{11}^{+}\Sigma_{12}$$

On obtient immédiatement l'expression classique et historique :

$$\Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12}$$

Conclusion :

Le formalisme basé sur l'inverse de Moore-Penrose est une **généralisation robuste** : il garantit que la théorie de la projection linéaire et du complément de Schur fonctionne sans division par zéro même si la matrice est singulière (présence de colinéarité). Lorsque le système est sain et strictement défini positif, il se réduit proprement au complément de Schur standard, ouvrant la voie royale vers l'algorithme de Cholesky.

Note: Pour être sur que de manière empirique le pseudo-inverse doit égal à l'inverse, il nous faut démontrer que la matrice de covariance est presque sûrement définie positive.

Théorème : Caractère presque sûrement défini positif de la matrice de covariance empirique

Soient $X^{(1)}, X^{(2)}, \ldots, X^{(T)}$ des observations i.i.d. d'un vecteur aléatoire de dimension p, issues d'une loi continue. Si le nombre d'observations est suffisant ($T \geq p + 1$), alors la matrice de covariance empirique $\hat{\Sigma}$ est **presque sûrement symétrique définie positive** :

$$\mathbb{P}\left(\hat{\Sigma} \succ 0\right) = 1$$

Démonstration (simplifiée)

Objectif : Montrer qu'il est impossible (de probabilité nulle) qu'il existe un vecteur de direction $v \neq 0$ pour lequel la variance empirique s'annule.

Étape 1 : Expression de la forme quadratique empirique

R rappelons la forme quadratique d'une matrice de covariance empirique $\hat{\Sigma}$ pour un vecteur déterministe non nul $v \in \mathbb{R}^p$:

$$v^T \hat{\Sigma} v = \frac{1}{T-1} \sum_{t=1}^{T} \left(v^T (X^{(t)} - \bar{X}) \right)^2$$

où $\bar{X}$ est la moyenne empirique des observations.

Étape 2 : Condition d'annulation

Pour que la matrice $\hat{\Sigma}$ ne soit *pas* définie positive, il faudrait qu'il existe un vecteur non nul $v \neq 0$ tel que la forme quadratique soit nulle :

$$v^T \hat{\Sigma} v = 0$$

Puisqu'une somme de carrés réels n'est composée que de termes positifs ou nuls, dire que le total est nul implique que **chaque terme individuel doit être nul** :

$$v^T (X^{(t)} - \bar{X}) = 0 \quad \text{pour tout } t = 1, \ldots, T$$

Étape 3 : Interprétation géométrique et contradiction probabiliste

1. L'expression $v^T (X^{(t)} - \bar{X}) = 0$ signifie géométriquement que **tous les points observés $X^{(t)}$ se situent exactement sur un même hyperplan** de dimension inférieure à p (défini par le vecteur normal v).
2. Or, par hypothèse, nos observations proviennent d'une loi de probabilité continue dans $\mathbb{R}^p$ (comme une loi normale non dégénérée). Dans un espace continu, la probabilité que des points aléatoires indépendants tombent par un pur hasard exactement sur un hyperplan de dimension inférieure est **strictement nulle**.
3. Par conséquent, l'existence d'un tel vecteur $v \neq 0$ annulant la forme quadratique est un événement de probabilité 0.

Étape 4 : Conclusion

Puisqu'aucun vecteur non nul v ne peut annuler la forme quadratique (sauf avec une probabilité de 0), nous avons pour tout $v \neq 0$:

$$v^T \hat{\Sigma} v > 0$$

La matrice $\hat{\Sigma}$ est donc **presque sûrement symétrique définie positive**. ■

Remarque de synthèse : Portée empirique et opérationnelle

Sous l'hypothèse standard où le nombre d'observations T excède la dimension du système ($T \geq p+1$), toute matrice de covariance empirique issue d'un processus continu est **presque sûrement symétrique définie positive** ($\mathbb{P}(\hat{\Sigma} \succ 0) = 1$).

Par conséquent, dans la grande majorité des applications empiriques classiques (où $T > p$), la condition de positivité stricte nécessaire à l'utilisation de la décomposition de Cholesky est naturellement validée de fait, sans qu'il soit requis de vérifier explicitement les conditions de positivité du complément de Schur.

Néanmoins, il convient de nuancer la portée de cet acquis empirique :

1. **Rôle opérationnel du complément de Schur :** S'il n'est plus nécessaire de le mobiliser comme un test de validation de la positivité (qui est acquise presque sûrement), le complément de Schur demeure l'outil de calcul fondamental pour expliciter les covariances résiduelles et construire les prédictions (espérances conditionnelles) par blocs.
2. **La limite des régimes de grande dimension ($p > T$) :** Dès lors que le nombre d'actifs dépasse l'historique d'observations disponible — un cas fréquent en finance quantitative moderne —, la matrice empirique perd sa stricte positivité et devient singulière. Le cadre généralisé (via le pseudoinverse et le complément de Schur généralisé) redevient alors indispensable pour éviter l'effondrement des modèles.

Exemples empiriques pour comprendre ce que l'on fait via Python et NumPy

Imaginons un portefeuille composé de **deux actifs financiers** (par exemple, deux actions) dont on observe les rendements quotidiens sur un historique de $T = 4$ **jours**.

Soit le vecteur aléatoire $X = \begin{pmatrix} X_1 \\ X_2 \end{pmatrix}$ représentant ces rendements. Voici nos observations $X^{(1)}$ à $X^{(4)}$:

- **Jour 1 :** $X^{(1)} = \begin{pmatrix} 1\% \\ 2\% \end{pmatrix}$

- **Jour 2 :** $X^{(2)} = \begin{pmatrix} 3\% \\ 1\% \end{pmatrix}$
- **Jour 3 :** $X^{(3)} = \begin{pmatrix} 2\% \\ 3\% \end{pmatrix}$
- **Jour 4 :** $X^{(4)} = \begin{pmatrix} 4\% \\ 4\% \end{pmatrix}$

---

1. Calcul de l'espérance empirique ($\hat{\mu}$)

L'espérance théorique $\mathbb{E}[X]$ est remplacée en empirique par la moyenne arithmétique de l'échantillon :

$$\hat{\mu} = \frac{1}{T} \sum_{t=1}^{T} X^{(t)}$$

- **Pour l'actif 1 (X_1) :** $\frac{1+3+2+4}{4} = \frac{10}{4} = 2.5\%$
- **Pour l'actif 2 (X_2) :** $\frac{2+1+3+4}{4} = \frac{10}{4} = 2.5\%$

Le vecteur des espérances empiriques est donc :

$$\hat{\mu} = \begin{pmatrix} 2.5 \\ 2.5 \end{pmatrix}$$

---

2. Centrage des données

Pour calculer la matrice de variance-covariance, on commence par centrer chaque observation en soustrayant la moyenne $\hat{\mu}$:

- $\tilde{X}^{(1)} = \begin{pmatrix} 1 - 2.5 \\ 2 - 2.5 \end{pmatrix} = \begin{pmatrix} -1.5 \\ -0.5 \end{pmatrix}$
- $\tilde{X}^{(2)} = \begin{pmatrix} 3 - 2.5 \\ 1 - 2.5 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -1.5 \end{pmatrix}$
- $\tilde{X}^{(3)} = \begin{pmatrix} 2 - 2.5 \\ 3 - 2.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix}$

- $\tilde{X}^{(4)} = \begin{pmatrix} 4 - 2.5 \\ 4 - 2.5 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix}$

---

3. Calcul de la matrice de variance-covariance empirique ($\hat{\Sigma}$)

Appliquons la formule empirique (avec le facteur de Bessel $T - 1 = 3$ pour l'absence de biais) :

$$\hat{\Sigma} = \frac{1}{T-1} \sum_{t=1}^{T} \tilde{X}^{(t)} (\tilde{X}^{(t)})^T$$

Calculons les produits externes pour chaque jour et sommons-les :

1. **Jour 1 :** $\begin{pmatrix} -1.5 \\ -0.5 \end{pmatrix} \begin{pmatrix} -1.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 2.25 & 0.75 \\ 0.75 & 0.25 \end{pmatrix}$
2. **Jour 2 :** $\begin{pmatrix} 0.5 \\ -1.5 \end{pmatrix} \begin{pmatrix} 0.5 & -1.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.75 \\ -0.75 & 2.25 \end{pmatrix}$
3. **Jour 3 :** $\begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \begin{pmatrix} -0.5 & 0.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}$
4. **Jour 4 :** $\begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix} \begin{pmatrix} 1.5 & 1.5 \end{pmatrix} = \begin{pmatrix} 2.25 & 2.25 \\ 2.25 & 2.25 \end{pmatrix}$

En additionnant ces quatre matrices terme à terme :

- **Somme de la diagonale (Variances brutes) :** $2.25 + 0.25 + 0.25 + 2.25 = 5.0$ pour l'actif 1, et $0.25 + 2.25 + 0.25 + 2.25 = 5.0$ pour l'actif 2.
- **Somme du hors-diagonale (Covariances brutes) :** $0.75 - 0.75 - 0.25 + 2.25 = 2.0$.

Divisons maintenant par $T - 1 = 3$ pour obtenir $\hat{\Sigma}$:

$$\hat{\Sigma} = \frac{1}{3} \begin{pmatrix} 5.0 & 2.0 \\ 2.0 & 5.0 \end{pmatrix} \approx \begin{pmatrix} 1.67 & 0.67 \\ 0.67 & 1.67 \end{pmatrix}$$

Lecture et interprétation financière de ce résultat :

- **Sur la diagonale (1.67) :** C'est la variance empirique de chaque actif. Ils ont exactement la même volatilité apparente dans cet échantillon.
- **Hors diagonale (0.67) :** C'est la covariance positive entre X_1 et X_2. Cela signifie que les deux actifs ont tendance à évoluer dans le même sens (quand l'un monte, l'autre monte généralement aussi).

Via NumPy, cela donne:

In [115…

```
X = np.array([
    [1,2],
    [3,1],
    [2,3],
    [4,4]
])
X_cov = np.cov(X,rowvar=False,ddof=1)
X_var = np.var(X,axis=0,ddof=1)
X_mean = np.mean(X,axis=0)


print("==="*15)

print(X_mean)

print("==="*15)

print(X_var)

print("==="*15)

print(X_cov)
print("==="*15)

print("The covariance matrix is definite positive." if np.linalg.det(X_cov) > 0 else "The covariance matrix is not
```

```
==========================================
[2.5 2.5]
==========================================
[1.66666667 1.66666667]
==========================================
[[1.66666667 0.66666667]
 [0.66666667 1.66666667]]
==========================================
The covariance matrix is definite positive.
```

Un autre exemple:

Pour construire une matrice de covariance qui soit **semi-définie positive** (c'est-à-dire singulière, avec un déterminant nul), il suffit de créer une situation de **colinéarité parfaite** entre nos variables.

Imaginons que l'actif X_2 soit une fonction linéaire exacte de l'actif X_1 (par exemple, $X_2 = 2X_1$). Reprenons $T = 4$ observations :

- **Jour 1 :** $X^{(1)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$
- **Jour 2 :** $X^{(2)} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$
- **Jour 3 :** $X^{(3)} = \begin{pmatrix} 3 \\ 6 \end{pmatrix}$
- **Jour 4 :** $X^{(4)} = \begin{pmatrix} 4 \\ 8 \end{pmatrix}$

---

1. Calcul de l'espérance empirique ($\hat{\mu}$)

- **Pour X_1 :** $\frac{1+2+3+4}{4} = 2.5$
- **Pour X_2 :** $\frac{2+4+6+8}{4} = 5.0$

$$\hat{\mu} = \begin{pmatrix} 2.5 \\ 5.0 \end{pmatrix}$$

---

2. Centrage des données

- $\tilde{X}^{(1)} = \begin{pmatrix} 1 - 2.5 \\ 2 - 5.0 \end{pmatrix} = \begin{pmatrix} -1.5 \\ -3.0 \end{pmatrix}$
- $\tilde{X}^{(2)} = \begin{pmatrix} 2 - 2.5 \\ 4 - 5.0 \end{pmatrix} = \begin{pmatrix} -0.5 \\ -1.0 \end{pmatrix}$
- $\tilde{X}^{(3)} = \begin{pmatrix} 3 - 2.5 \\ 6 - 5.0 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 1.0 \end{pmatrix}$
- $\tilde{X}^{(4)} = \begin{pmatrix} 4 - 2.5 \\ 8 - 5.0 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 3.0 \end{pmatrix}$

---

3. Calcul de la matrice de covariance empirique ($\hat{\Sigma}$)

En sommant les produits externes et en divisant par $T - 1 = 3$:

$$\hat{\Sigma} = \begin{pmatrix} \frac{5}{3} & \frac{10}{3} \\ \frac{10}{3} & \frac{20}{3} \end{pmatrix} \approx \begin{pmatrix} 1.67 & 3.33 \\ 3.33 & 6.67 \end{pmatrix}$$

Via NumPy, cela donne:

In [120…

```python
import numpy as np

X_singular = np.array([
    [1, 2],
    [2, 4],
    [3, 6],
    [4, 8]
])


X_singular_cov = np.cov(X_singular,rowvar=False,ddof=1)
X_singular_var = np.var(X_singular,axis=0,ddof=1)
X_singular_mean = np.mean(X_singular,axis=0)
```

```python
print("==="*15)

print(X_singular_mean)

print("==="*15)

print(X_singular_var)

print("==="*15)

print(X_singular_cov)
print("==="*15)

print("The covariance matrix is definite positive." if np.linalg.det(X_singular_cov) > 0 else "The covariance matri
```

```
=============================================
[2.5 5. ]
=============================================
[1.66666667 6.66666667]
=============================================
[[1.66666667 3.33333333]
 [3.33333333 6.66666667]]
=============================================
The covariance matrix is not definite positive.
```

Reprenons nos deux exemples fondamentaux sous un angle comparatif direct. Nous allons appliquer à la fois **l'inverse classique** (Σ^{-1}) et **le pseudoinverse de Moore-Penrose** (Σ^{+}) sur nos deux cas : le cas régulier (défini positif) et le cas singulier (semi-défini positif).

Cas 1 : La matrice de covariance définie positive (Cas sain)

Reprenons notre premier exemple de matrice de covariance empirique exacte (en écriture fractionnaire pour la clarté algébrique) :

$$\Sigma_1 = \frac{1}{3}\begin{pmatrix} 5 & 2 \\ 2 & 5 \end{pmatrix}$$

1. Approche algébrique

- **Calcul du déterminant :**

$$\det(\Sigma_1) = \left(\frac{1}{3}\right)^2 \times (5 \times 5 - 2 \times 2) = \frac{1}{9} \times (25 - 4) = \frac{21}{9} = \frac{7}{3} \neq 0$$

Puisque le déterminant est non nul, **l'inverse classique existe**.

- **L'inverse classique (Σ_1^{-1}) :**

$$\Sigma_1^{-1} = 3 \times \frac{1}{21} \begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix} = \frac{1}{7} \begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix}$$

- **Le pseudoinverse de Moore-Penrose (Σ_1^+) :** Comme Σ_1 est de plein rang (ses deux valeurs propres sont strictement positives), le pseudoinverse coïncide exactement avec l'inverse classique :

$$\Sigma_1^+ = \Sigma_1^{-1} = \frac{1}{7} \begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix}$$

In [129…

```python
import numpy as np

print("--- CASE 1 : Definite positive ---")
# Clasical inverse
inv_1 = np.linalg.inv(X_cov)

print("Clasical inverse (inv) :\n", np.round(inv_1, 4))

# Pseudoinverse
pinv_1 = np.linalg.pinv(Sigma_1)
print("Pseudoinverse (pinv) :\n", np.round(pinv_1, 4))
```

```
--- CASE 1 : Definite positive ---
Clasical inverse (inv) :
 [[ 0.7143 -0.2857]
 [-0.2857  0.7143]]
Pseudoinverse (pinv) :
 [[ 0.7143 -0.2857]
 [-0.2857  0.7143]]
```

Cas 2 : La matrice de covariance semi-définie positive (Cas singulier)

Reprenons notre second exemple où les variables étaient parfaitement colinéaires ($X_2 = 2X_1$) :

$$\Sigma_2 = \frac{1}{3}\begin{pmatrix} 5 & 10 \\ 10 & 20 \end{pmatrix}$$

1. Approche algébrique

- **Calcul du déterminant :**

$$\det(\Sigma_2) = \left(\frac{1}{3}\right)^2 \times (5 \times 20 - 10 \times 10) = \frac{1}{9} \times (100 - 100) = 0$$

Le déterminant est nul. **La matrice n'est pas inversible.**

- **L'inverse classique (Σ_2^{-1}) : N'existe pas.** Algébriquement, cela conduirait à diviser par le déterminant (donc par zéro).
- **Le pseudoinverse de Moore-Penrose (Σ_2^{+}) :** Passons par la décomposition spectrale de Σ_2. La trace vaut $\frac{25}{3}$ et le déterminant 0, ce qui donne les valeurs propres $\lambda_1 = \frac{25}{3}$ et $\lambda_2 = 0$.
- Le vecteur propre associé à λ_1 est normalisé, et celui de λ_2 capture la direction de la colinéarité.
- En appliquant la règle du pseudoinverse (on inverse λ_1 en $\frac{3}{25}$ et on laisse $\lambda_2 = 0$ intact), on obtient :

$$\Sigma_2^{+} = \frac{3}{125}\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$$

(Remarque : On peut vérifier que $\Sigma_2 \Sigma_2^{+} \Sigma_2 = \Sigma_2$, ce qui valide l'une des conditions de Penrose).

2. Implémentation en Python

Synthèse de l'expérience :

- **Dans le Cas 1 (Sain) :** `np.linalg.inv` et `np.linalg.pinv` donnent exactement le même résultat. C'est le monde idéal où l'on se trouve presque toujours en pratique (grâce au théorème de positivité presque sûre).
- **Dans le Cas 2 (Colinéaire) :** `np.linalg.inv` échoue et génère une erreur critique, tandis que `np.linalg.pinv` neutralise la dimension nulle et fournit un opérateur de projection stable.

In [130…

```python
print("\n--- CASE 2: Positive Semi-Definite (Singular) ---")

# 1. Attempting standard inverse
try:
    inv_2 = np.linalg.inv(X_singular_cov)
    print("Standard inverse: successful", inv_2)
except np.linalg.LinAlgError as e:
    print("Standard inverse (inv): Error caught ->", e)

# 2. Using the pseudoinverse
pinv_2 = np.linalg.pinv(X_singular_cov)
print("Pseudoinverse (pinv): Successfully calculated!\n", np.round(pinv_2, 4))
```

```
--- CASE 2: Positive Semi-Definite (Singular) ---
Standard inverse (inv): Error caught -> Singular matrix
Pseudoinverse (pinv): Successfully calculated!
 [[0.024 0.048]
 [0.048 0.096]]
```

Reprenons notre analyse comparative sur la base de nos deux matrices 2×2 (Σ_1 et Σ_2), en veillant à appliquer la notation rigoureuse du pseudoinverse de Moore-Penrose (Σ^+) dès que la structure s'inscrit dans un cadre généralisé.

Application au Cas 1 : Matrice définie positive (Cas sain)

Dans ce cas, notre matrice de covariance est :

$$\Sigma_1 = \frac{1}{3}\begin{pmatrix} 5 & 2 \\ 2 & 5 \end{pmatrix}$$

1. Modèle de régression OLS (Estimation de X_2 sachant X_1)

Pour estimer X_2 à partir de X_1, on calcule le coefficient de régression (la pente B) via l'inverse (qui coïncide ici avec le pseudoinverse) de la variance de X_1 ($\Sigma_{11} = \frac{5}{3}$) :

$$B_1 = \Sigma_{21}\Sigma_{11}^{-1} = \left(\frac{2}{3}\right) \times \left(\frac{3}{5}\right) = \frac{2}{5} = 0.4$$

L'équation de prédiction centrée s'écrit donc :

$$\hat{X}_2 = 0.4 \times X_1$$

2. Variance résiduelle (Complément de Schur S)

L'incertitude qui subsiste après cette prédiction est :

$$S_1 = \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12} = \frac{5}{3} - \left(\frac{2}{3} \times \frac{3}{5} \times \frac{2}{3}\right) = \frac{5}{3} - \frac{4}{15} = 1.4$$

In [137…

```python
import numpy as np

# --- CASE 1: Positive Definite Matrix ---
Sigma_1 = (1/3) * np.array([
    [5, 2],
    [2, 5]
])

# Partitioning to predict variable 1 (index 1) from variable 0 (index 0)
Sigma_11_1 = Sigma_1[0, 0]
Sigma_12_1 = Sigma_1[0, 1]
Sigma_21_1 = Sigma_1[1, 0]
Sigma_22_1 = Sigma_1[1, 1]

# Slope B (using standard inverse / division by scalar)
B_1 = Sigma_21_1 / Sigma_11_1
# Schur complement (residual variance)
Schur_1 = Sigma_22_1 - Sigma_21_1 * (1.0 / Sigma_11_1) * Sigma_12_1

print("--- CASE 1 : Positive Definite ---")
```

```
print(f"Regression coefficient B: {B_1}")
print(f"Residual variance (Schur S): {Schur_1}")
```

```
--- CASE 1 : Positive Definite ---
Regression coefficient B: 0.4
Residual variance (Schur S): 1.4
```

Application au Cas 2 : Matrice semi-définie positive (Cas colinéaire)

Dans ce cas, notre matrice de covariance est :

$$\Sigma_2 = \frac{1}{3}\begin{pmatrix} 5 & 10 \\ 10 & 20 \end{pmatrix}$$

1. Modèle de régression OLS (Estimation de X_2 sachant X_1)

Puisque la matrice globale Σ_2 est singulière, nous adoptons rigoureusement la notation du pseudoinverse Σ^+ pour caractériser l'opérateur de projection.

Ici, X_1 a pour variance $\Sigma_{11} = \frac{5}{3}$ et la covariance est $\Sigma_{21} = \frac{10}{3}$. Le coefficient de régression (pente) s'exprime via le pseudoinverse du bloc explicatif :

$$B_2 = \Sigma_{21}\Sigma_{11}^{+} = \left(\frac{10}{3}\right) \times \left(\frac{3}{5}\right) = 2$$

(Ce qui est parfaitement cohérent puisque nous avions défini $X_2 = 2X_1$).

L'équation de prédiction est donc :

$$\hat{X}_2 = 2 \times X_1$$

2. Variance résiduelle (Complément de Schur S)

Calculons le complément de Schur associé en utilisant cette même formulation généralisée :

$$S_2 = \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{+}\Sigma_{12} = \frac{20}{3} - \left(\frac{10}{3} \times \frac{3}{5} \times \frac{10}{3}\right) = \frac{20}{3} - \frac{20}{3} = 0$$

- **Interprétation :** La variance résiduelle est rigoureusement nulle ($S_2 = 0$). La prédiction de X_2 par X_1 est **parfaite**, sans aucune incertitude, traduisant une colinéarité absolue.
- **Remarque :** Ces résultats mathématiques illustrent un cas d'école purement déterministe. On comprend ainsi pourquoi, dans la réalité empirique, une relation strictement linéaire n'existe jamais : le bruit inhérent aux données génère toujours un complément de Schur strictement positif ($S > 0$), garantissant la stricte positivité de la matrice.

In [138…

```python
# --- CASE 2: Positive Semi-Definite (Collinear) ---
Sigma_2 = (1/3) * np.array([
    [5, 10],
    [10, 20]
])

Sigma_11_2 = Sigma_2[0, 0]
Sigma_12_2 = Sigma_2[0, 1]
Sigma_21_2 = Sigma_2[1, 0]
Sigma_22_2 = Sigma_2[1, 1]

# Slope B (Here the submatrix Sigma_11 is a non-zero scalar, but if it were of higher dimension, we would use np.li
B_2 = Sigma_21_2 / Sigma_11_2
Schur_2 = Sigma_22_2 - Sigma_21_2 * (1.0 / Sigma_11_2) * Sigma_12_2

print("\n--- CASE 2: Positive Semi-Definite (Collinear) ---")
print(f"Regression coefficient B: {B_2}")
print(f"Residual variance (Schur S): {Schur_2} (Exactly 0)")
```

```
--- CASE 2: Positive Semi-Definite (Collinear) ---
Regression coefficient B: 2.0
Residual variance (Schur S): 0.0 (Exactly 0)
```

Remarquons qu'enfin, le complément de Schur n'est rien d'autre qu'une manière d'obtenir le coeffient R^2 du modèle OLS.

Calculons donc maintenant précisément ce **ratio de variation non expliquée** (S/Σ_{22}) pour nos deux cas. Ce ratio représente la proportion de la variance totale de X_2 qui échappe au modèle OLS. Son complément à 1 ($1 - S/\Sigma_{22}$) n'est autre que le fameux **coefficient de détermination (R^2)**.

Application au Cas 1 : Matrice définie positive (Cas sain)

Dans ce cas sain, rappelons nos valeurs exactes :

- Variance totale de X_2 : $\Sigma_{22} = \frac{5}{3}$
- Variance résiduelle (Complément de Schur) : $S_1 = \frac{7}{5} = 1.4$

1. Approche algébrique du ratio

Le pourcentage de variation non expliquée par la régression est :

$$\text{Ratio non expliqué} = \frac{S_1}{\Sigma_{22}} = \frac{\frac{7}{5}}{\frac{5}{3}} = \frac{7}{5} \times \frac{3}{5} = \frac{21}{25} = 0.84 \text{ (soit } 84\%)$$

Par conséquent, la part de variance expliquée (R^2) est :

$$R^2 = 1 - 0.84 = 0.16 \text{ (soit } 16\%)$$

2. Implémentation en Python

In [142…

```python
import numpy as np

# --- CASE 1: Positive Definite Matrix ---
X_case1 = np.array([
    [1, 2],
    [3, 1],
    [2, 3],
    [4, 4]
])

Sigma_1 = np.cov(X_case1, ddof=1, rowvar=False)
```

```python
Sigma_11_1 = Sigma_1[0, 0]
Sigma_12_1 = Sigma_1[0, 1]
Sigma_21_1 = Sigma_1[1, 0]
Sigma_22_1 = Sigma_1[1, 1]

# 1. R^2 calculation via Schur complement (residual variance)
Schur_1 = Sigma_22_1 - Sigma_21_1 * (1.0 / Sigma_11_1) * Sigma_12_1
unexplained_ratio_1 = Schur_1 / Sigma_22_1
r_squared_schur_1 = 1.0 - unexplained_ratio_1

# 2. R^2 calculation via np.corrcoef (Pearson correlation squared)
corr_matrix_1 = np.corrcoef(X_case1, rowvar=False)
pearson_r_1 = corr_matrix_1[0, 1]
r_squared_corr_1 = pearson_r_1 ** 2

print("--- CASE 1 : Positive Definite ---")
print(f"Explained variance ratio (R^2 via Schur): {r_squared_schur_1:.4f}")
print(f"R^2 via np.corrcoef (r^2): {r_squared_corr_1:.4f}")
print(f"Are both methods equal? {np.isclose(r_squared_schur_1, r_squared_corr_1)}")
```

```
--- CASE 1 : Positive Definite ---
Explained variance ratio (R^2 via Schur): 0.1600
R^2 via np.corrcoef (r^2): 0.1600
Are both methods equal? True
```

Application au Cas 2 : Matrice semi-définie positive (Cas colinéaire)

Dans ce cas pathologique de colinéarité parfaite ($X_2 = 2X_1$) :

- Variance totale de X_2 : $\Sigma_{22} = \frac{20}{3}$
- Variance résiduelle (Complément de Schur) : $S_2 = 0$

1. Approche algébrique du ratio

Le pourcentage de variation non expliquée par la régression s'annule :

$$\text{Ratio non expliqué} = \frac{S_2}{\Sigma_{22}} = \frac{0}{\frac{20}{3}} = 0 \text{ (soit } 0\%)$$

La part de variance expliquée atteint son maximum absolu :

$$R^2 = 1 - 0 = 1 \text{ (soit } 100\%)$$

In [143…

```python
# --- CASE 2: Positive Semi-Definite (Collinear) ---
X_case2 = np.array([
    [1, 2],
    [2, 4],
    [3, 6],
    [4, 8]
])

Sigma_2 = np.cov(X_case2, ddof=1, rowvar=False)
Sigma_11_2 = Sigma_2[0, 0]
Sigma_12_2 = Sigma_2[0, 1]
Sigma_21_2 = Sigma_2[1, 0]
Sigma_22_2 = Sigma_2[1, 1]

# 1. R^2 calculation via Schur complement
Schur_2 = Sigma_22_2 - Sigma_21_2 * (1.0 / Sigma_11_2) * Sigma_12_2
unexplained_ratio_2 = Schur_2 / Sigma_22_2
r_squared_schur_2 = 1.0 - unexplained_ratio_2

# 2. R^2 calculation via np.corrcoef (Pearson correlation squared)
corr_matrix_2 = np.corrcoef(X_case2, rowvar=False)
pearson_r_2 = corr_matrix_2[0, 1]
r_squared_corr_2 = pearson_r_2 ** 2

print("\n--- CASE 2 : Positive Semi-Definite (Collinear) ---")
print(f"Explained variance ratio (R^2 via Schur): {r_squared_schur_2:.4f}")
print(f"R^2 via np.corrcoef (r^2): {r_squared_corr_2:.4f}")
print(f"Are both methods equal? {np.isclose(r_squared_schur_2, r_squared_corr_2)}")
```

```
--- CASE 2 : Positive Semi-Definite (Collinear) ---
Explained variance ratio (R^2 via Schur): 1.0000
R^2 via np.corrcoef (r^2): 1.0000
Are both methods equal? True
```

Finalement, testons la contrainte de Cholesky sur nos deux jeux de données:

Rappelons l'objectif de Cholesky : étant donnée une matrice de covariance Σ, on cherche à trouver une matrice triangulaire inférieure L telle que :

$$\Sigma = LL^T$$

Voyons ce qui se passe dans nos deux cas.

Cas 1 : La matrice de covariance définie positive (Cas Sain)

Pour notre matrice saine :

$$\Sigma_1 = \frac{1}{3}\begin{pmatrix} 5 & 2 \\ 2 & 5 \end{pmatrix}$$

1. Approche algébrique

Puisque Σ_1 est strictement définie positive ($\det(\Sigma_1) = \frac{7}{3} > 0$), la décomposition de Cholesky **réussit de manière fluide**. On cherche $L_1 = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$ tel que $L_1 L_1^T = \Sigma_1$:

- $l_{11} = \sqrt{\Sigma_{11}} = \sqrt{\frac{5}{3}}$
- $l_{21} = \frac{\Sigma_{21}}{l_{11}} = \frac{2}{3} \times \sqrt{\frac{3}{5}} = \frac{2}{\sqrt{15}}$
- $l_{22} = \sqrt{\Sigma_{22} - l_{21}^2} = \sqrt{\frac{5}{3} - \frac{4}{15}} = \sqrt{\frac{21}{15}} = \sqrt{\frac{7}{5}}$

2. Implémentation en Python

In [146…

```python
import numpy as np

# --- CASE 1: Positive Definite Matrix ---
Sigma_1 = (1/3) * np.array([
    [5, 2],
    [2, 5]
])

try:
    # Cholesky decomposition
    L_1 = np.linalg.cholesky(Sigma_1)
    print("--- CASE 1 : Cholesky Decomposition ---")
    print("Success! Lower triangular matrix L_1:\n", np.round(L_1, 4))

    # Verification: L_1 @ L_1.T should equal Sigma_1
    reconstructed_sigma_1 = L_1 @ L_1.T
    print("Reconstruction check passed:", np.allclose(reconstructed_sigma_1, Sigma_1))

except np.linalg.LinAlgError as e:
    print("Cholesky failed:", e)
```

```
--- CASE 1 : Cholesky Decomposition ---
Success! Lower triangular matrix L_1:
 [[1.291  0.    ]
 [0.5164 1.1832]]
Reconstruction check passed: True
```

Cas 2 : La matrice semi-définie positive / colinéaire (`Sigma_2`)

Pour notre matrice colinéaire :

$$\Sigma_2 = \frac{1}{3}\begin{pmatrix} 5 & 10 \\ 10 & 20 \end{pmatrix}$$

1. Approche algébrique et obstacle

Puisque Σ_2 est singulière ($\det(\Sigma_2) = 0$), elle n'est **pas** définie positive (ses valeurs propres sont $\frac{25}{3}$ et 0).

- Rappel : le complément de Schur de cette matrice était égal à **0** ($S_2 = 0$).
- Algébriquement, lors de l'algorithme de Cholesky, on essaie de calculer $l_{22} = \sqrt{\Sigma_{22} - l_{21}^2}$, ce qui correspond exactement à la racine carrée du complément de Schur !
- Obtenir $S_2 = 0$ signifie qu'on essaie de faire la racine carrée de 0, ou pire, si de légères erreurs d'arrondi machine surviennent, on pourrait se retrouver avec un nombre négatif. **L'algorithme de Cholesky plante donc systématiquement.**

2. Implémentation en Python (et gestion de l'erreur)

In [148…

```python
# --- CASE 2: Positive Semi-Definite (Collinear) ---
Sigma_2 = (1/3) * np.array([
    [5, 10],
    [10, 20]
])

try:
    # Attempting Cholesky decomposition on a singular matrix
    L_2 = np.linalg.cholesky(Sigma_2)
    print("\n--- CASE 2 : Cholesky Decomposition ---")
    print("Success! L_2:\n", L_2)
except np.linalg.LinAlgError as e:
    print("\n--- CASE 2 : Cholesky Decomposition ---")
    print("Cholesky FAILED as expected -> Error caught:", e)
    print("Reason: The matrix is singular (Schur complement S = 0). Cholesky requires strict positive-definiteness.
```

```
--- CASE 2 : Cholesky Decomposition ---
Cholesky FAILED as expected -> Error caught: Matrix is not positive definite
Reason: The matrix is singular (Schur complement S = 0). Cholesky requires strict positive-definiteness.
```

Comment contourner cela en pratique financière ?

En finance quantitative, il arrive fréquemment qu'un grand univers d'actifs (par exemple, des centaines d'actions ou de taux) présente des colinéarités (redondance d'information, secteurs ultra-corrélés).

Pour pouvoir quand même générer des trajectoires via Cholesky, on utilise une technique de régularisation appelée le **Jittering** (ou régularisation Ridge) : on rajoute un bruit infime sur la diagonale pour rendre la matrice strictement définie positive :

$$\Sigma_{\text{reg}} = \Sigma_2 + \epsilon I$$

Où ϵ est un nombre minuscule (ex: 10^{-8}).

In [150…

```python
import numpy as np

# --- CASE 2: Positive Semi-Definite (Collinear) with Jitter ---
Sigma_2 = (1/3) * np.array([
    [5, 10],
    [10, 20]
])

# Define a small jitter epsilon
epsilon = 1e-8

# Add epsilon to the diagonal to make the matrix strictly positive definite
Sigma_2_reg = Sigma_2 + epsilon * np.eye(2)

try:
    # Cholesky decomposition on the regularized matrix
    L_2_reg = np.linalg.cholesky(Sigma_2_reg)
    print("--- CASE 2 : Cholesky with Jitter (Regularization) ---")
    print("Success! Lower triangular matrix L_2_reg:\n", np.round(L_2_reg, 6))

    # Verification: L_2_reg @ L_2_reg.T should be extremely close to Sigma_2
    reconstructed_sigma_2 = L_2_reg @ L_2_reg.T
    print("Reconstruction check passed:", np.allclose(reconstructed_sigma_2, Sigma_2, atol=1e-4))

except np.linalg.LinAlgError as e:
    print("Cholesky failed even with jitter:", e)
```

```
--- CASE 2 : Cholesky with Jitter (Regularization) ---
Success! Lower triangular matrix L_2_reg:
 [[1.290994e+00 0.000000e+00]
 [2.581989e+00 2.240000e-04]]
Reconstruction check passed: True
```

In [154…

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Génération d'un jeu de données synthétique (3 variables)
np.random.seed(42)
n_samples = 600

# Matrice de covariance théorique globale [X1, X2, Z]
cov_matrix = np.array([
    [4.0, 2.0, 3.0],
    [2.0, 3.0, 1.5],
    [3.0, 1.5, 3.5]
])

data = np.random.multivariate_normal([0, 0, 0], cov_matrix, size=n_samples)
X1 = data[:, 0]
X2 = data[:, 1]
Z  = data[:, 2:3] # Variable de conditionnement

# 2. Partitionnement de la matrice et calcul du Complément de Schur
A = cov_matrix[:2, :2]  # Covariance de [X1, X2]
B = cov_matrix[:2, 2:]  # Covariance entre [X1, X2] et Z
C = cov_matrix[2:, :2]  # Transposée de B
D = cov_matrix[2:, 2:]  # Variance de Z

# Calcul théorique du Complément de Schur S = A - B * inv(D) * C
schur_complement = A - B @ np.linalg.inv(D) @ C

# 3. Calcul des résidus (Régression de [X1, X2] sur Z)
X_vars = np.column_stack([X1, X2])
beta = np.linalg.inv(Z.T @ Z) @ (Z.T @ X_vars)
residuals = X_vars - (Z @ beta)

print("Complément de Schur théorique (S) :\n", schur_complement)
print("Covariance empirique des résidus :\n", np.cov(residuals, rowvar=False))

# 4. Visualisation avec Matplotlib
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Panneau 1 : Données originales
```

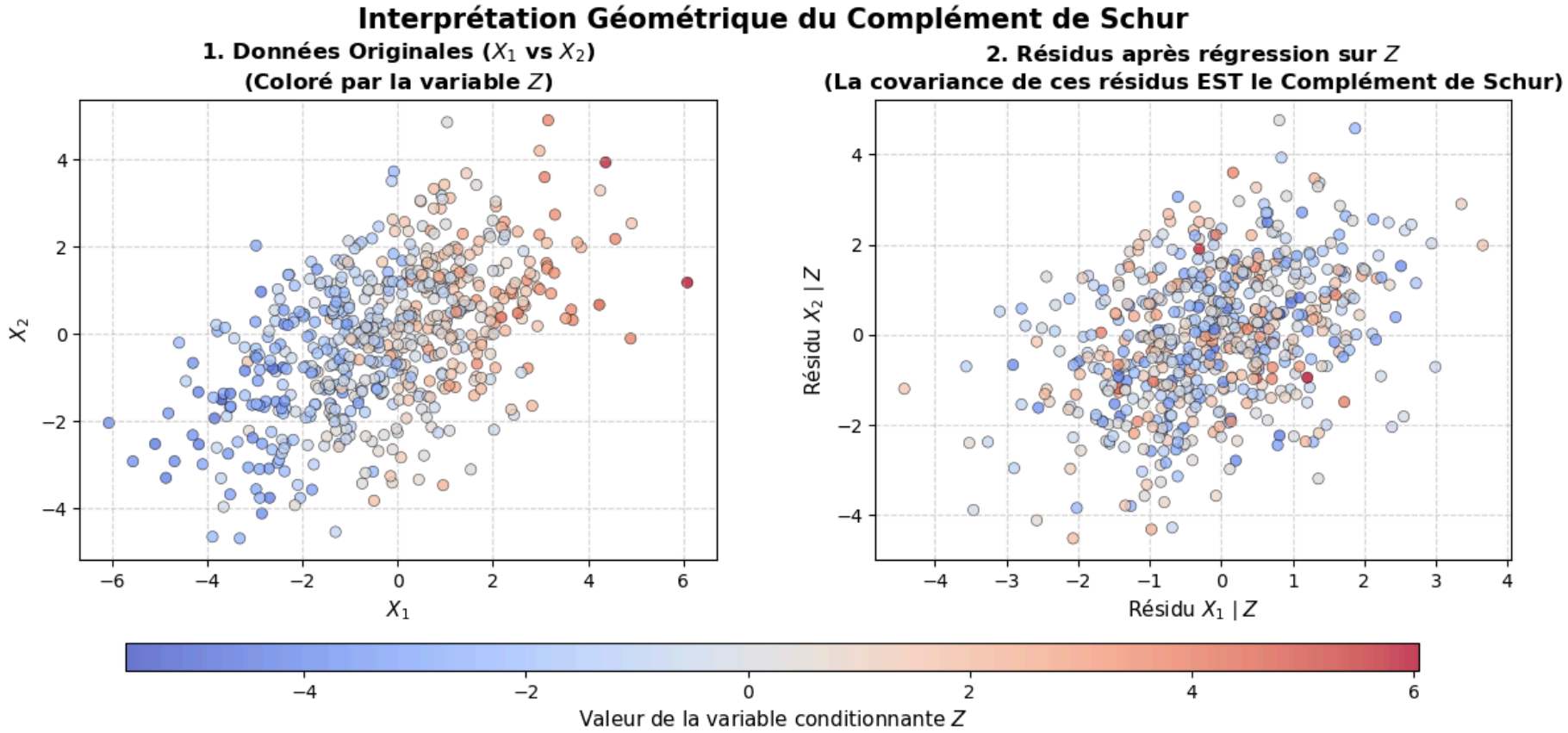
```python
sc1 = axes[0].scatter(X1, X2, c=Z.flatten(), cmap='coolwarm', alpha=0.75, edgecolors='k', linewidth=0.4)
axes[0].set_title('1. Données Originales ($X_1$ vs $X_2$)\n(Coloré par la variable $Z$)', fontsize=12, fontweight='
axes[0].set_xlabel('$X_1$', fontsize=11)
axes[0].set_ylabel('$X_2$', fontsize=11)
axes[0].grid(True, linestyle='--', alpha=0.5)

# Panneau 2 : Résidus (Complément de Schur)
sc2 = axes[1].scatter(residuals[:, 0], residuals[:, 1], c=Z.flatten(), cmap='coolwarm', alpha=0.75, edgecolors='k',
axes[1].set_title('2. Résidus après régression sur $Z$\n(La covariance de ces résidus EST le Complément de Schur)',
axes[1].set_xlabel('Résidu $X_1 \\mid Z$', fontsize=11)
axes[1].set_ylabel('Résidu $X_2 \\mid Z$', fontsize=11)
axes[1].grid(True, linestyle='--', alpha=0.5)

# Mise en page de la barre de couleur
fig.subplots_adjust(top=0.80, bottom=0.22, wspace=0.25)
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.035])
cbar = fig.colorbar(sc2, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Valeur de la variable conditionnante $Z$', fontsize=11)

fig.suptitle("Interprétation Géométrique du Complément de Schur", fontsize=15, fontweight='bold', y=0.92)
plt.show()
```

```
Complément de Schur théorique (S) :
 [[1.42857143 0.71428571]
 [0.71428571 2.35714286]]
Covariance empirique des résidus :
 [[1.41446662 0.688468  ]
 [0.688468   2.2956617 ]]
```



In [156…

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# 1. Génération de données synthétiques
np.random.seed(42)
n_samples = 500

Z = np.random.normal(0, 2, size=(n_samples, 1))
X1 = 1.5 * Z.flatten() + np.random.normal(0, 1, n_samples)
X2 = 1.2 * Z.flatten() + np.random.normal(0, 1, n_samples)
X3 = -1.0 * Z.flatten() + np.random.normal(0, 1, n_samples)
X_raw = np.column_stack([X1, X2, X3])
```

```python
# --- CAS 1 : ACP SANS correction (données brutes) ---
pca_raw = PCA(n_components=2)
X_pca_raw = pca_raw.fit_transform(X_raw)
var_raw = pca_raw.explained_variance_ratio_ * 100  # En pourcentages

# --- CAS 2 : ACP AVEC correction (Résidus / Schur) ---
beta = np.linalg.inv(Z.T @ Z) @ (Z.T @ X_raw)
X_residuals = X_raw - (Z @ beta)

pca_clean = PCA(n_components=2)
X_pca_clean = pca_clean.fit_transform(X_residuals)
var_clean = pca_clean.explained_variance_ratio_ * 100  # En pourcentages

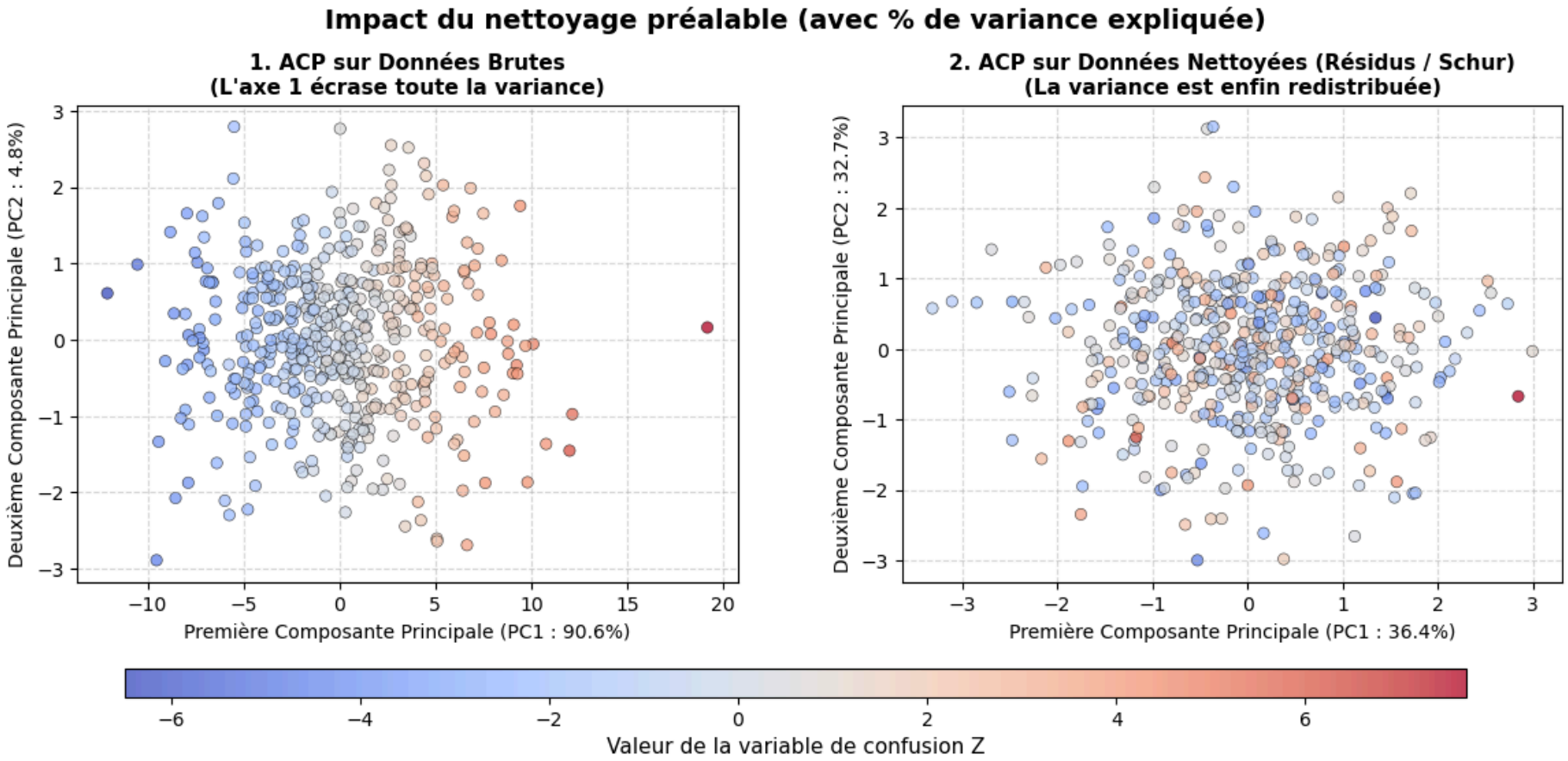
# 2. Visualisation comparative avec les pourcentages sur les axes
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Graphique de gauche : ACP Brute
sc1 = axes[0].scatter(X_pca_raw[:, 0], X_pca_raw[:, 1], c=Z.flatten(), cmap='coolwarm', alpha=0.75, edgecolors='k',
axes[0].set_title('1. ACP sur Données Brutes\n(L\'axe 1 écrase toute la variance)', fontsize=11, fontweight='bold')
axes[0].set_xlabel(f'Première Composante Principale (PC1 : {var_raw[0]:.1f}%)', fontsize=10)
axes[0].set_ylabel(f'Deuxième Composante Principale (PC2 : {var_raw[1]:.1f}%)', fontsize=10)
axes[0].grid(True, linestyle='--', alpha=0.5)

# Graphique de droite : ACP Nettoyée
sc2 = axes[1].scatter(X_pca_clean[:, 0], X_pca_clean[:, 1], c=Z.flatten(), cmap='coolwarm', alpha=0.75, edgecolors=
axes[1].set_title('2. ACP sur Données Nettoyées (Résidus / Schur)\n(La variance est enfin redistribuée)', fontsize=
axes[1].set_xlabel(f'Première Composante Principale (PC1 : {var_clean[0]:.1f}%)', fontsize=10)
axes[1].set_ylabel(f'Deuxième Composante Principale (PC2 : {var_clean[1]:.1f}%)', fontsize=10)
axes[1].grid(True, linestyle='--', alpha=0.5)

# Barre de couleur globale
fig.subplots_adjust(top=0.80, bottom=0.22, wspace=0.25)
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.035])
cbar = fig.colorbar(sc2, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Valeur de la variable de confusion Z', fontsize=11)

fig.suptitle("Impact du nettoyage préalable (avec % de variance expliquée)", fontsize=14, fontweight='bold', y=0.92
plt.show()
```



Chapitre 3: La loi normale multi-variée:

Définition

Un vecteur aléatoire $X = (X_1, X_2, \ldots, X_p)^T$ de dimension p suit une **loi normale multivariée** (ou gaussienne multidimensionnelle), notée :

$$X \sim \mathcal{N}(\mu, \Sigma)$$

lorsque toute combinaison linéaire de ses composantes suit une loi normale univariée.

Il est entièrement caractérisé par deux paramètres :

- **Le vecteur des moyennes ($\mu \in \mathbb{R}^p$) :**

$$\mu = \mathbb{E}[X]$$

- **La matrice de covariance ($\Sigma \in \mathbb{R}^{p \times p}$) :** Une matrice symétrique et semi-définie positive ($\Sigma \succeq 0$) qui quantifie la dispersion et les interdépendances linéaires entre les variables.

Dans le cas régulier où Σ est strictement définie positive ($\Sigma \succ 0$), sa fonction de densité de probabilité (PDF) est donnée par :

$$f(x) = \frac{1}{(2\pi)^{p/2}\sqrt{\det(\Sigma)}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$$

In [190…

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal
import pandas as pd

# 1. Configuration initiale
np.random.seed(42)
n_samples = 1000
mu = np.array([0, 0])

# Définition des 6 cas
cases = [
    {"name": "Cas 1: Variances égales, rho = 0 (Cercle)", "var1": 2.0, "var2": 2.0, "rho": 0.0},
    {"name": "Cas 2: Variances égales, rho > 0", "var1": 2.0, "var2": 2.0, "rho": 0.75},
    {"name": "Cas 3: Variances égales, rho < 0", "var1": 2.0, "var2": 2.0, "rho": -0.75},
    {"name": "Cas 4: Variances différentes, rho = 0", "var1": 4.0, "var2": 1.0, "rho": 0.0},
    {"name": "Cas 5: Variances différentes, rho > 0", "var1": 4.0, "var2": 1.0, "rho": 0.75},
    {"name": "Cas 6: Variances différentes, rho < 0", "var1": 4.0, "var2": 1.0, "rho": -0.75}
]

fig, axes = plt.subplots(3, 2, figsize=(14, 18))
axes = axes.flatten()

table_data = []
```

```python
lim_min, lim_max = -5.5, 5.5  # Limites d'axes communes et strictes

# 2. Boucle sur les 6 cas
for i, case in enumerate(cases):
    v1 = case["var1"]
    v2 = case["var2"]
    rho = case["rho"]

    sigma1 = np.sqrt(v1)
    sigma2 = np.sqrt(v2)
    cov12 = rho * sigma1 * sigma2

    # Matrice de covariance
    cov_matrix = np.array([
        [v1, cov12],
        [cov12, v2]
    ])

    # Génération des données synthétiques
    data = np.random.multivariate_normal(mu, cov_matrix, size=n_samples)
    X1 = data[:, 0]
    X2 = data[:, 1]

    # Calcul théorique rigoureux de la droite OLS (X2 sur X1)
    # beta_1 = Cov(X1, X2) / Var(X1) = (rho * sigma1 * sigma2) / sigma1^2 = rho * (sigma2 / sigma1)
    beta_1 = rho * (sigma2 / sigma1)
    beta_0 = 0.0

    # Stockage des résultats pour le tableau
    table_data.append({
        "Cas": f"Cas {i+1}",
        "Description": case["name"],
        "Variance 1 (sigma1^2)": v1,
        "Variance 2 (sigma2^2)": v2,
        "Corrélation (rho)": rho,
        "Pente théorique (beta_1)": round(beta_1, 4),
        "Ordonnée (beta_0)": round(beta_0, 4)
    })

    # Grille pour les contours
    xx, yy = np.meshgrid(np.linspace(lim_min, lim_max, 100), np.linspace(lim_min, lim_max, 100))
```

```
    pos = np.dstack((xx, yy))
    rv = multivariate_normal(mu, cov_matrix)
    zz = rv.pdf(pos)

    # 3. Tracé des graphiques
    ax = axes[i]
    ax.scatter(X1, X2, alpha=0.3, color='royalblue', edgecolor='none', s=20, label='Données')
    contour = ax.contour(xx, yy, zz, levels=6, cmap='viridis')
    ax.clabel(contour, inline=True, fontsize=8, fmt='%.3f')

    # Tracé de la droite OLS théorique en pointillés rouges
    x_line = np.linspace(lim_min, lim_max, 100)
    y_line = beta_0 + beta_1 * x_line
    ax.plot(x_line, y_line, color='crimson', linestyle='--', linewidth=2.5, label='Droite OLS théorique ($X_2$ sur

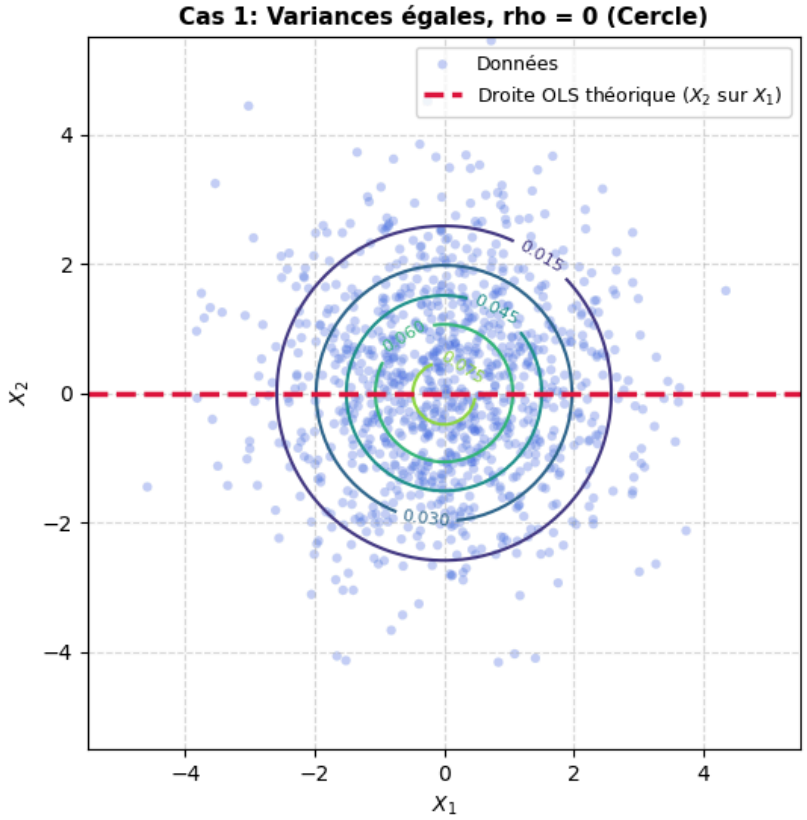
    # Paramètres d'affichage stricts
    ax.set_xlim(lim_min, lim_max)
    ax.set_ylim(lim_min, lim_max)
    ax.set_aspect('equal', adjustable='box')

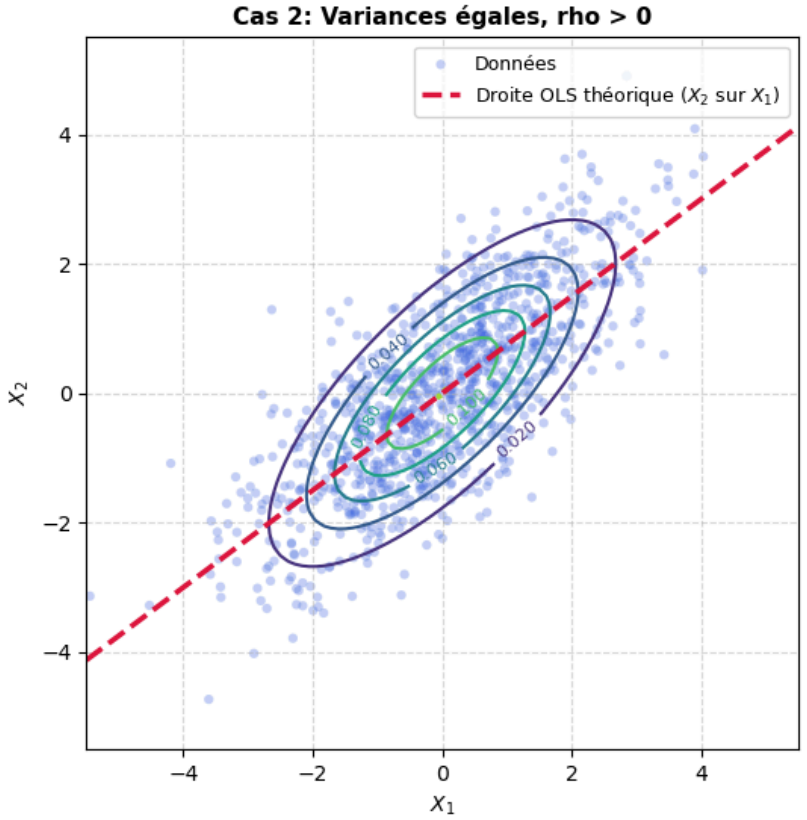
    ax.set_title(case["name"], fontsize=11, fontweight='bold')
    ax.set_xlabel('$X_1$', fontsize=10)
    ax.set_ylabel('$X_2$', fontsize=10)
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.legend(loc='upper right', fontsize=9)

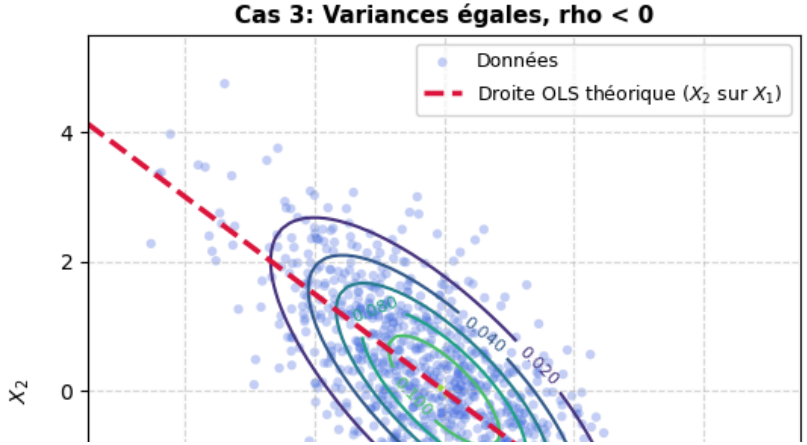
# Mise en page globale
plt.suptitle("Lois Multinormales Bivariées et Droites OLS Théoriques (Échelles unifiées)", fontsize=14, fontweight=
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

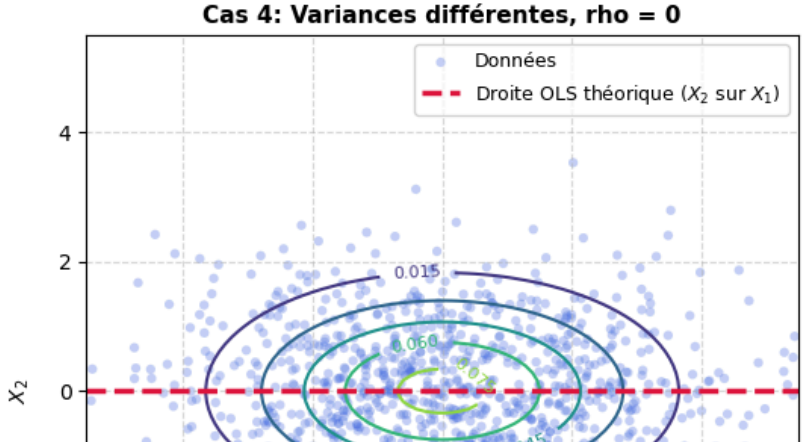
# 4. Affichage du tableau récapitulatif
df_results = pd.DataFrame(table_data)
print("\n" + "="*85)
print(" TABLEAU RÉCAPITULATIF DES COEFFICIENTS DE RÉGRESSION THÉORIQUES (OLS)")
print("="*85)
print(df_results.to_string(index=False))
print("="*85 + "\n")
```

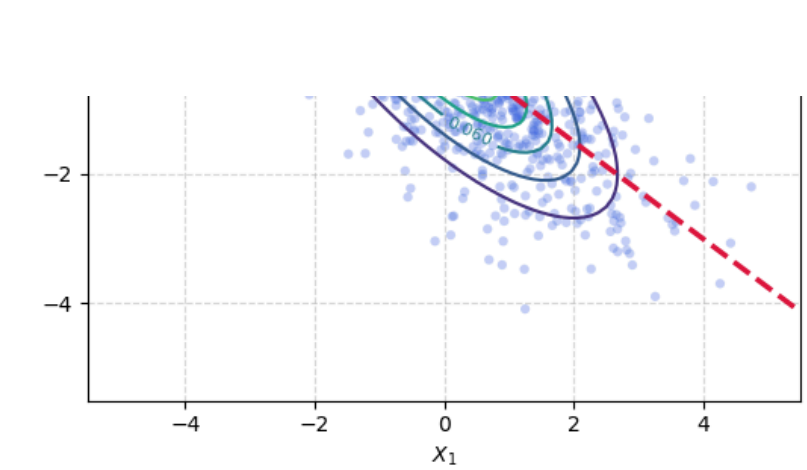
Lois Multinormales Bivariées et Droites OLS Théoriques (Échelles unifiées)

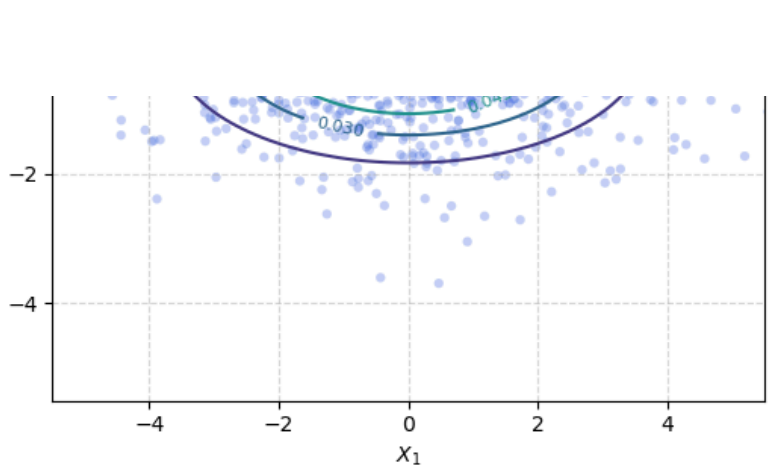


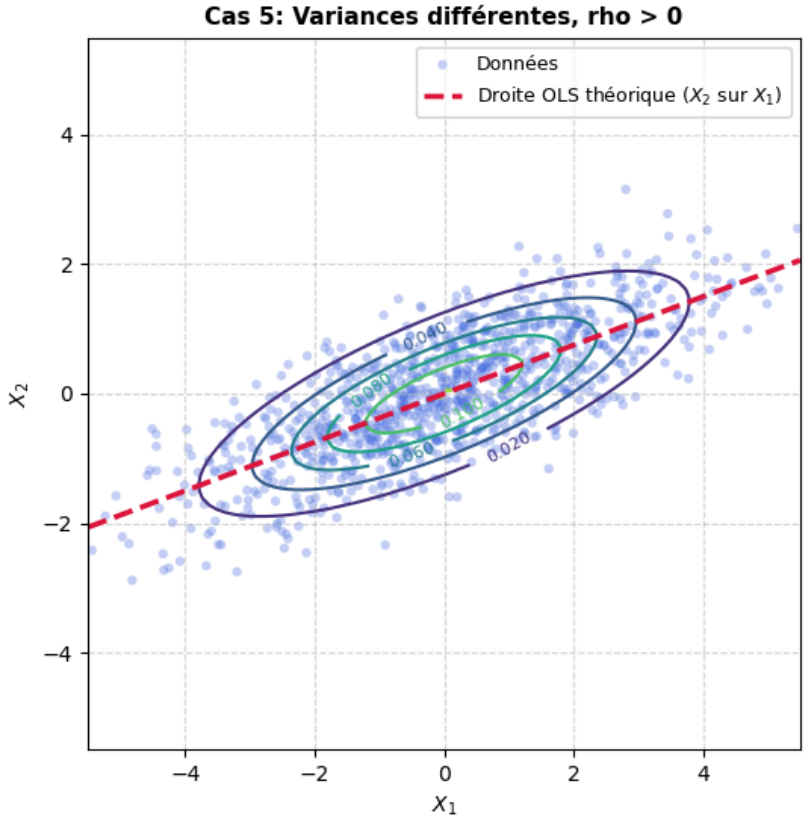


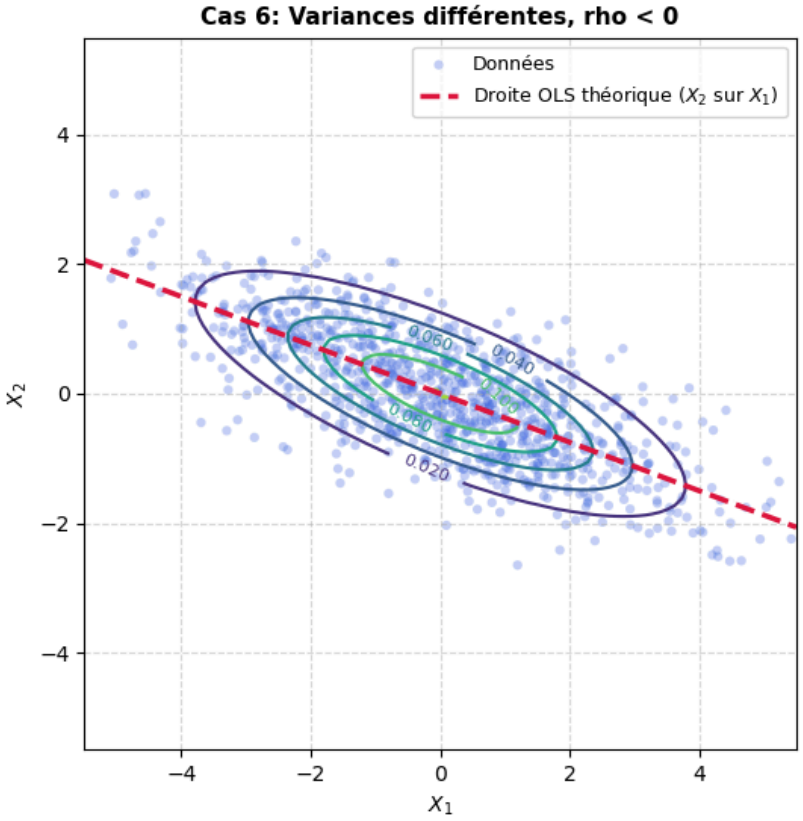





0.060
−2
−4
−4
−2
0
2
4
X_1


0.030
−2
−4
−4
−2
0
2
4
X_1


Cas 5: Variances différentes, rho > 0
Données
Droite OLS théorique (X_2 sur X_1)
0.040
0.080
0.100
0.060
0.020
4
2
0
−2
−4
X_2
−4
−2
0
2
4
X_1


Cas 6: Variances différentes, rho < 0
Données
Droite OLS théorique (X_2 sur X_1)
0.060
0.040
0.080
0.020
4
2
0
−2
−4
X_2
−4
−2
0
2
4
X_1

```
================================================================================
 TABLEAU RÉCAPITULATIF DES COEFFICIENTS DE RÉGRESSION THÉORIQUES (OLS)
================================================================================
  Cas                                Description  Variance 1 (sigma1^2)  Variance 2 (sigma2^2)  Corrélation (rho)  Pe
nte théorique (beta_1)  Ordonnée (beta_0)
Cas 1 Cas 1: Variances égales, rho = 0 (Cercle)                    2.0                    2.0               0.00
0.000                0.0
Cas 2          Cas 2: Variances égales, rho > 0                    2.0                    2.0               0.75
0.750                0.0
Cas 3          Cas 3: Variances égales, rho < 0                    2.0                    2.0              -0.75
-0.750               0.0
Cas 4     Cas 4: Variances différentes, rho = 0                    4.0                    1.0               0.00
0.000                0.0
Cas 5     Cas 5: Variances différentes, rho > 0                    4.0                    1.0               0.75
0.375                0.0
Cas 6     Cas 6: Variances différentes, rho < 0                    4.0                    1.0              -0.75
-0.375               0.0
================================================================================
```

Théorème : Transformation affine d'un vecteur gaussien

Soit X un vecteur aléatoire de dimension p suivant une loi normale multivariée :

$$X \sim \mathcal{N}(\mu, \Sigma)$$

Soit Y un nouveau vecteur aléatoire de dimension q obtenu par une **transformation affine** de X :

$$Y = AX + b$$

où :

- A est une **matrice de dimensions** $q \times p$ (qui réalise la transformation linéaire, sans qu'il soit nécessaire qu'elle soit carrée ou inversible),
- b est un **vecteur constant de dimension** q (qui réalise la translation).

Alors, le vecteur Y suit **encore et toujours une loi normale multivariée** de dimension q, dont les nouveaux paramètres sont donnés par :

$$Y \sim \mathcal{N}\left(A\mu + b,\ A\Sigma A^T\right)$$

> **Note de rigueur géométrique :**
>
> - Si A est de rang plein (et $q \leq p$), la matrice de covariance $A\Sigma A^T$ est inversible et la loi de Y est à densité (non dégénérée).
> - Si A n'est pas de rang plein (par exemple, si elle projette l'espace vers une dimension inférieure $q < p$ en écrasant des axes), la matrice $A\Sigma A^T$ devient singulière (son déterminant est nul). La loi de Y est alors **dégénérée** (le nuage de points est aplati sur un sous-espace), mais sa structure probabiliste demeure rigoureusement gaussienne.

In [193…

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# 1. Configuration initiale de la loi normale de base X ~ N(mu, Sigma)
np.random.seed(42)
n_samples = 1000
mu = np.array([0, 0])

v1, v2, rho = 4.0, 1.0, 0.5
sigma1, sigma2 = np.sqrt(v1), np.sqrt(v2)
cov12 = rho * sigma1 * sigma2
Sigma = np.array([[v1, cov12], [cov12, v2]])

# Génération des données de base X
X = np.random.multivariate_normal(mu, Sigma, size=n_samples)

# --- Transformation 1 : NON DÉGÉNÉRÉE (Matrice A1 inversible) ---
A1 = np.array([[1.5, -0.5],
               [0.5,  2.0]])
b1 = np.array([1.0, -1.0])
Y_non_deg = (A1 @ X.T).T + b1
mu_non_deg = A1 @ mu + b1
Sigma_non_deg = A1 @ Sigma @ A1.T
```

```
# --- Transformation 2 : DÉGÉNÉRÉE (Matrice A2 non de rang plein, det = 0) ---
A2 = np.array([[1.0, 1.0],
               [0.5, 0.5]])
Y_deg = (A2 @ X.T).T
mu_deg = A2 @ mu
Sigma_deg = A2 @ Sigma @ A2.T  # Déterminant = 0 (singulière)

# Grilles pour les contours
lim = 7
xx, yy = np.meshgrid(np.linspace(-lim, lim, 100), np.linspace(-lim, lim, 100))
pos = np.dstack((xx, yy))

# 2. Tracé de la figure à 3 panneaux
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# --- GRAPHIQUE 1 : Loi de base X ---
ax1 = axes[0]
ax1.scatter(X[:, 0], X[:, 1], alpha=0.3, color='royalblue', s=20, label='Données $X$')
rv1 = multivariate_normal(mu, Sigma)
contour1 = ax1.contour(xx, yy, rv1.pdf(pos), levels=5, cmap='viridis')
ax1.clabel(contour1, inline=True, fontsize=8, fmt='%.3f')

beta_1_base = cov12 / v1
x_line = np.linspace(-lim, lim, 100)
ax1.plot(x_line, beta_1_base * x_line, color='crimson', linestyle='--', linewidth=2.5, label='Droite OLS')

ax1.set_title("1. Loi de base $X \\sim \\mathcal{N}(\\mu, \\Sigma)$\n(Espace initial 2D, $\\det(\\Sigma) > 0$)", fo
ax1.set_xlabel('$X_1$')
ax1.set_ylabel('$X_2$')
ax1.set_aspect('equal')
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.legend(loc='upper right', fontsize=8)
ax1.set_xlim(-lim, lim)
ax1.set_ylim(-lim, lim)


# --- GRAPHIQUE 2 : Cas Non Dégénéré (Y = A1*X + b) ---
ax2 = axes[1]
ax2.scatter(Y_non_deg[:, 0], Y_non_deg[:, 1], alpha=0.3, color='forestgreen', s=20, label='Données $Y = A_1X + b$')
rv2 = multivariate_normal(mu_non_deg, Sigma_non_deg)
```

```
contour2 = ax2.contour(xx, yy, rv2.pdf(pos), levels=5, cmap='viridis')
ax2.clabel(contour2, inline=True, fontsize=8, fmt='%.3f')

beta_1_nd = Sigma_non_deg[0, 1] / Sigma_non_deg[0, 0]
beta_0_nd = mu_non_deg[1] - beta_1_nd * mu_non_deg[0]
ax2.plot(x_line, beta_0_nd + beta_1_nd * x_line, color='crimson', linestyle='--', linewidth=2.5, label='Droite OLS'

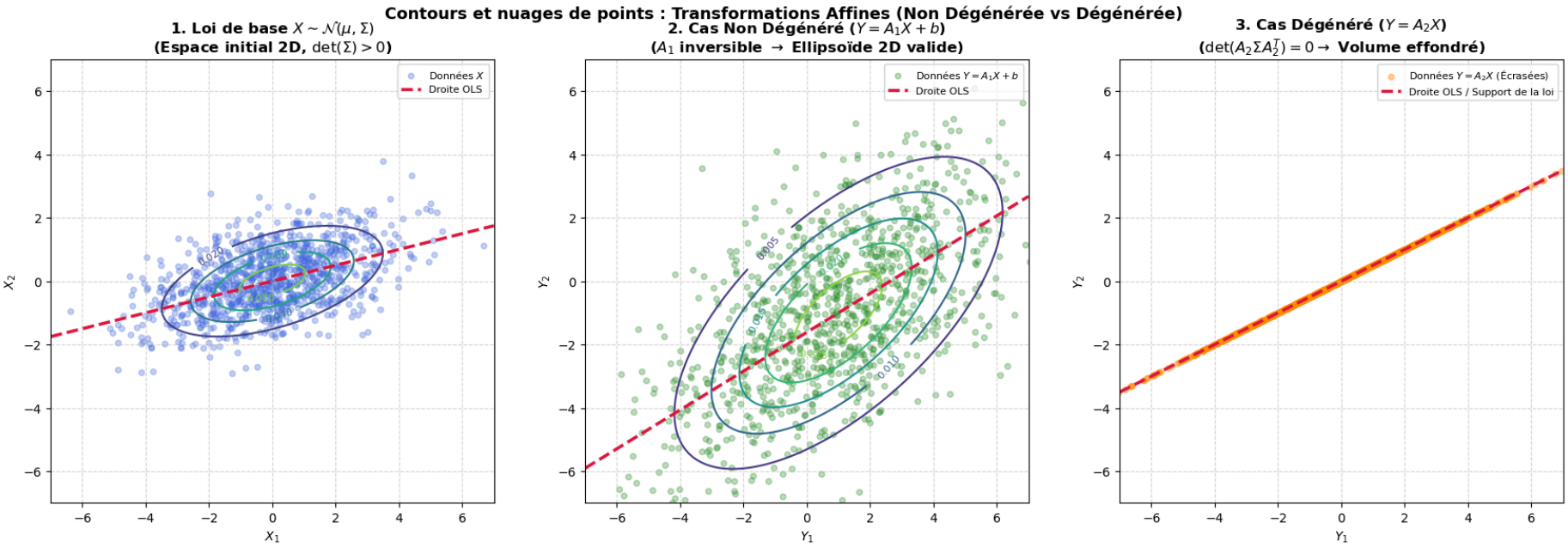
ax2.set_title("2. Cas Non Dégénéré ($Y = A_1X + b$)\n($A_1$ inversible $\\rightarrow$ Ellipsoïde 2D valide)", fontw
ax2.set_xlabel('$Y_1$')
ax2.set_ylabel('$Y_2$')
ax2.set_aspect('equal')
ax2.grid(True, linestyle='--', alpha=0.5)
ax2.legend(loc='upper right', fontsize=8)
ax2.set_xlim(-lim, lim)
ax2.set_ylim(-lim, lim)


# --- GRAPHIQUE 3 : Cas Dégénéré (Y = A2*X) ---
ax3 = axes[2]
ax3.scatter(Y_deg[:, 0], Y_deg[:, 1], alpha=0.4, color='darkorange', s=20, label='Données $Y = A_2X$ (Écrasées)')

# Pas de contour 2D possible car det(Sigma_deg) = 0 (matrice de covariance singulière)
# On trace la droite sur laquelle tout le volume s'est effondré
ax3.plot(x_line, 0.5 * x_line, color='crimson', linestyle='--', linewidth=2.5, label='Droite OLS / Support de la lo

ax3.set_title("3. Cas Dégénéré ($Y = A_2X$)\n($\\det(A_2\\Sigma A_2^T) = 0 \\rightarrow$ Volume effondré)", fontwei
ax3.set_xlabel('$Y_1$')
ax3.set_ylabel('$Y_2$')
ax3.set_aspect('equal')
ax3.grid(True, linestyle='--', alpha=0.5)
ax3.legend(loc='upper right', fontsize=8)
ax3.set_xlim(-lim, lim)
ax3.set_ylim(-lim, lim)

# Mise en page globale
plt.suptitle("Contours et nuages de points : Transformations Affines (Non Dégénérée vs Dégénérée)", fontsize=13, fo
plt.tight_layout()
plt.show()
```



Définition

La **loi conditionnelle d'une variable normale multivariée en dimension** n est la distribution de probabilité d'un sous-ensemble de variables aléatoires sachant qu'un autre sous-ensemble a pris des valeurs fixes et observées.

Si l'on considère un vecteur global $X \in \mathbb{R}^n$ suivant une loi normale multivariée $X \sim \mathcal{N}(\mu, \Sigma)$, partitionné en deux blocs :

- $X_1 \in \mathbb{R}^q$ (les variables observées),
- $X_2 \in \mathbb{R}^{n-q}$ (les variables à prédire),

Plus précisément, si l'on associe directement les sous-vecteurs X_1 et X_2 aux colonnes (ou au ligne car c'est symétriquement pareil en fait) :

$$\begin{array}{l} \\ \begin{cases} X_1 \ (\dim q) \\ X_2 \ (\dim n-q) \end{cases} \end{array} \quad \begin{array}{c} \overbrace{}^{X_1} \quad \overbrace{}^{X_2} \\ \underset{\dim q}{} \quad \underset{\dim n-q}{} \\ \left(\begin{array}{c|c} \Sigma_{11} & \Sigma_{12} \\ \hline \Sigma_{21} & \Sigma_{22} \end{array}\right) \end{array}$$

Lecture de cette matrice par blocs :

- Lignes du haut (X_1) et colonnes de gauche (X_1) $\rightarrow$ Donnent le bloc Σ_{11} (la variance interne des variables observées).
- Lignes du bas (X_2) et colonnes de droite (X_2) $\rightarrow$ Donnent le bloc Σ_{22} (la variance interne des variables à prédire).
- Lignes du bas (X_2) et colonnes de gauche (X_1) $\rightarrow$ Donnent le bloc Σ_{21} (le lien de covariance entre les deux groupes).

Propriétés fondamentales de la loi conditionnelle

- **Nature de la loi :** La loi conditionnelle de X_2 sachant $X_1 = x_1$ demeure **strictement une loi normale multivariée** de dimension $n - q$, notée $X_2 \mid (X_1 = x_1) \sim \mathcal{N}(\mu_{2|1}, \Sigma_{2|1})$.
- **Espérance conditionnelle ($\mu_{2|1}$) :**

$$\mu_{2|1} = \mu_2 + \Sigma_{21}\Sigma_{11}^{-1}(x_1 - \mu_1)$$

- Ce terme généralise l'ajustement par régression linéaire (OLS) en dimension n, où $\Sigma_{21}\Sigma_{11}^{-1}$ joue le rôle de la matrice des coefficients de régression.

- **Variance conditionnelle ($\Sigma_{2|1}$) :**

$$\Sigma_{21} = \Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12}$$

- Connue sous le nom de **complément de Schur**, cette matrice quantifie la réduction de l'incertitude (le volume de l'ellipse) apportée par la connaissance des variables X_1.

Chapitre 4: Modèle du mouvement brownien géométrique pour un portefeuille multivarié.

Position du problème

Dans le cadre de la modélisation financière multi-actifs (considérant un panier de n actifs), l'objectif est de décrire l'évolution conjointe et stochastique des prix $S_1(t), S_2(t), \dots, S_n(t)$. Pour bâtir ce cadre rigoureusement, la modélisation ne peut pas démarrer directement avec des dynamiques corrélées complexes ; elle doit s'ancrer sur un socle de sources d'aléas fondamentales et indépendantes.

- **Le socle des bruits indépendants :** On introduit un système de n mouvements browniens standard et indépendants, notés $W_1(t), W_2(t), \dots, W_n(t)$.
- **Propriétés statistiques de base :** Chaque processus $W_j(t)$ représente une source de risque élémentaire du marché. À tout instant $t \geq 0$, il suit une loi normale centrée dont la variance est proportionnelle au temps :

$$W_j(t) \sim \mathcal{N}(0, t)$$

Leurs incréments infinitésimaux vérifient ainsi $dW_j(t) \sim \mathcal{N}(0, dt)$. De plus, pour tout couple d'indices distincts $j \neq k$, les mouvements browniens sont indépendants, garantissant une covariance nulle entre ces bruits bruts.

- **Le défi de la modélisation :** Sur les marchés réels, les actifs financiers ne sont pas indépendants ; ils subissent des co-mouvements (tendances sectorielles, chocs macroéconomiques partagés). Le problème fondamental réside donc dans la capacité à transformer mathématiquement ce vecteur de bruits indépendants $W = (W_1, \dots, W_n)^T$ en un ensemble de chocs corrélés cohérents avec une matrice de covariance empirique donnée, tout en garantissant la stricte positivité des prix des actifs.

Passage aux chocs corrélés : Structure de la matrice de covariance (Σ)

Pour traduire mathématiquement les interdépendances et les co-mouvements observés entre les différents actifs d'un marché financier, le modèle repose sur l'introduction d'une **matrice de covariance** Σ de taille $n \times n$.

1. La décomposition de Cholesky (Du bruit indépendant au choc corrélé)

Puisque les ordinateurs et les théoriciens partent de bruits indépendants $W(t)$, il est nécessaire de "lier" ces bruits pour y injecter la matrice de covariance du marché. On utilise pour cela la **décomposition de Cholesky** de la matrice Σ :

$$\Sigma = LL^T$$

où L est une matrice triangulaire inférieure de dimension $n \times n$.

On définit alors le vecteur des chocs browniens corrélés $dZ(t)$ par la transformation linéaire suivante :

$$dZ(t) = L\, dW(t)$$

2. Expression pour un actif i du panier

En développant ce produit matriciel ligne par ligne, le choc aléatoire spécifique subit par un actif i s'écrit sous la forme d'une combinaison linéaire des n bruits indépendants du marché :

$$dZ_i(t) = \sum_{j=1}^{n} L_{ij}\, dW_j(t)$$

où les coefficients L_{ij} (souvent notés σ_{ij}) représentent la sensibilité de l'actif i au j-ème facteur de risque indépendant.

3. Conséquence sur la variance instantanée

En raison de l'indépendance des incréments $dW_j(t)$ (qui vérifient $\mathbb{E}[dW_j\, dW_k] = 0$ si $j \neq k$ et dt si $j = k$), la variance instantanée du choc $dZ_i(t)$ se simplifie directement à partir des coefficients de Cholesky :

$$\mathrm{Var}(dZ_i(t)) = \left(\sum_{j=1}^{n} L_{ij}^2 \right) dt = \Sigma_{ii}\, dt$$

L'élément diagonal Σ_{ii} de la matrice de covariance correspond ainsi rigoureusement à la **variance instantanée globale** de l'actif i, intégrant à la fois sa volatilité propre et son exposition à l'ensemble des facteurs de risque du marché.

Dynamique des actifs : Équations Différentielles Stochastiques (EDS)

À partir des chocs browniens corrélés $dZ_i(t)$, la dynamique de chaque actif $S_i(t)$ ($i = 1, \ldots, n$) au sein du portefeuille est modélisée par un mouvement brownien géométrique multi-actifs.

L'Équation Différentielle Stochastique (EDS) décrivant l'évolution de l'actif i s'écrit sous deux formes équivalentes :

- **Forme compacte (avec les chocs corrélés) :**

$$dS_i(t) = S_i(t)\mu_i\,dt + S_i(t)\,dZ_i(t)$$

- **Forme explicite (avec les bruits indépendants de base) :**

$$dS_i(t) = S_i(t)\mu_i\,dt + S_i(t)\sum_{j=1}^{n}\sigma_{ij}\,dW_j(t)$$

Où :

- μ_i représente le taux de rendement instantané espéré (la dérive ou *drift*) de l'actif i.
- σ_{ij} (ou les éléments de la matrice de Cholesky L_{ij}) caractérisent la sensibilité de l'actif i aux différents facteurs de risque du marché.

Application du Lemme d'Itô et solution explicite

Pour résoudre cette EDS et obtenir la formule fermée du prix $S_i(t)$, on applique le lemme d'Itô à la fonction de transformation $f(S_i) = \ln(S_i)$.

1. Calcul du terme de correction quadratique

Le lemme d'Itô pour $f(S_i) = \ln(S_i)$ introduit la dérivée seconde et la variation quadratique de l'actif $(dS_i(t))^2$:

$$(dS_i(t))^2 = S_i(t)^2\,(dZ_i(t))^2 = S_i(t)^2\Sigma_{ii}\,dt$$

où Σ_{ii} est l'élément diagonal de la matrice de covariance, représentant la variance instantanée globale des log-rendements de l'actif i.

2. L'EDS du log-prix

En injectant ce terme dans la formule d'Itô, on obtient l'équation différentielle du log-prix :

$$d(\ln S_i(t)) = \left(\mu_i - \frac{1}{2}\Sigma_{ii}\right) dt + dZ_i(t)$$

3. Solution explicite du modèle

En intégrant cette équation entre l'instant initial 0 et l'instant t, on obtient la solution explicite du prix de l'actif i :

$$S_i(t) = S_i(0) \exp\left(\left(\mu_i - \frac{1}{2}\Sigma_{ii}\right) t + \sum_{j=1}^{n} \sigma_{ij} W_j(t)\right)$$

In [479…

```python
import yfinance as yf

tickers = ["AAPL","NVDA","MSFT"]
start_date = "2026-04-04"
end_date = "2026-10-04"
data = yf.download(tickers, start_date,end_date, progress=False, auto_adjust=True)['Close']
```

In [505…

```python
import numpy as np
import numpy.random as rd

rd.seed(42)

dt = 1/252
days_left = 27

brownian_motions = np.array([rd.normal(loc=0,scale=np.sqrt(dt),size=days_left) for k in range(len(data.columns))])
log_returns = np.log((data/data.shift(1)).dropna())
cov_matrix = np.cov(log_returns,rowvar=False,ddof = 1)
cholesky_decomposition = np.linalg.cholesky(cov_matrix)

Z = cholesky_decomposition @ brownian_motions
mu = np.mean(log_returns,axis=0).values*dt
sigma_ii = np.diag(cov_matrix)*dt
S_0 = data.iloc[-1,:].values[:,None]

temps_cumule = np.arange(1, days_left + 1) * dt
GBM = S_0 * np.exp((mu - 1/2*sigma_ii)[:,None]*temps_cumule + np.cumsum(Z,axis=1))
```

In [506…

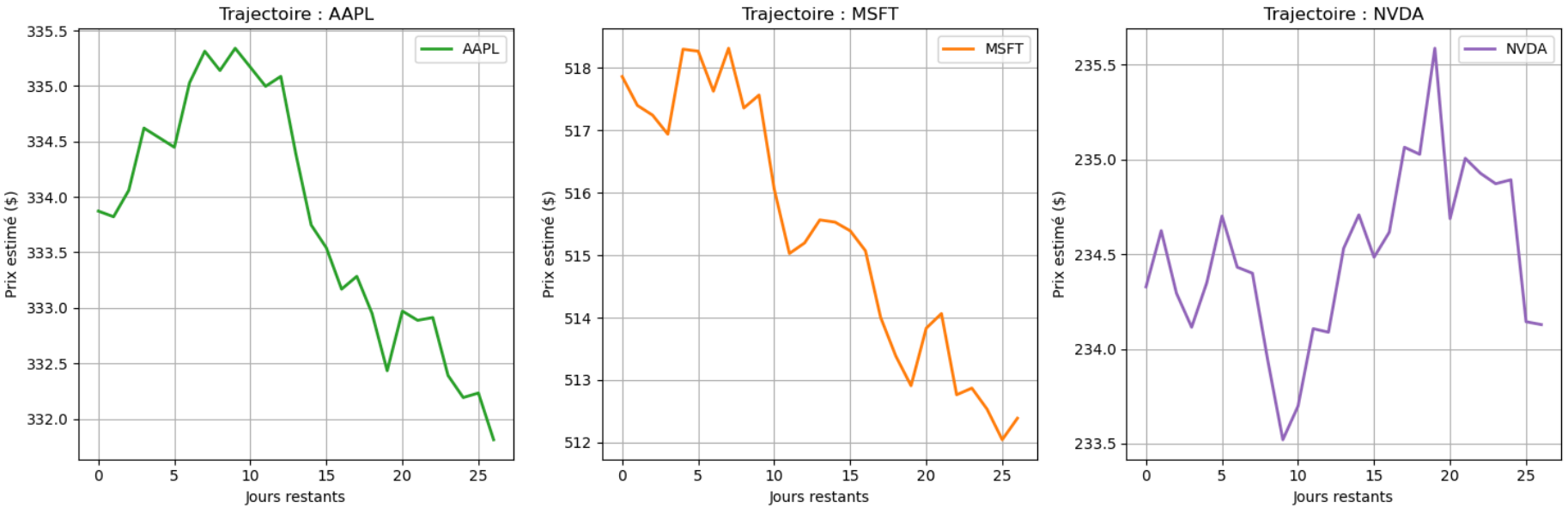
```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Définis une liste de couleurs pour tes 3 actifs (par exemple : vert, orange, violet)
couleurs = ["#2ca02c", "#ff7f0e", "#9467bd"]

for i, (ax, ticker, couleur) in enumerate(zip(axes, data.columns, couleurs)):
  ax.plot(
      GBM[i], color=couleur, linewidth=2,label = ticker
  )  # linewidth=2 rend la courbe un peu plus épaisse
  ax.set_title(f"Trajectoire : {ticker}")
  ax.set_xlabel("Jours restants")
  ax.set_ylabel("Prix estimé ($)")
  ax.grid(True)
  ax.legend()
```

```
plt.tight_layout()
plt.show()
```



Simulation de Monte-Carlo

In [493…

```
import numpy as np

tickers = data.columns

n_samples = 5000
stocks_nb = 3
days_left = 27
dt = 1/252

# 1. Extraction des données initiales et des rendements
S_0 = data.iloc[-1, :].values.reshape(1, stocks_nb, 1)  # Forme (1, 3, 1) pour le broadcast
returns = (data / data.shift(1)).dropna()
log_returns = np.log(returns).values

# 2. Paramètres statistiques
cov_matrix = np.cov(log_returns, rowvar=False, ddof=1)
```

```
mu = np.mean(log_returns, axis=0).reshape(1, stocks_nb, 1) * dt
sigma_ii = np.diag(cov_matrix).reshape(1, stocks_nb, 1) * dt

# 3. Décomposition de Cholesky
cholesky_decomposition = np.linalg.cholesky(cov_matrix)  # Forme (3, 3)

# 4. Génération de tous les chocs aléatoires d'un coup
# Forme : (n_samples, stocks_nb, days_left)
# On génère des normales centrées réduites, multipliées par sqrt(dt)
W = np.random.normal(0, np.sqrt(dt), size=(n_samples, stocks_nb, days_left))

# Application de la matrice de Cholesky sur l'axe des actifs
# np.matmul gère automatiquement les dimensions par lot (batch matrix multiplication)
Z = np.matmul(cholesky_decomposition, W)

# Somme cumulée sur les jours (axe -1)
cumulated_bm = np.cumsum(Z, axis=-1)

# 5. Calcul de la dérive (drift)
day_increments = np.arange(1, days_left + 1).reshape(1, 1, days_left)
drift = (mu - 0.5 * sigma_ii) * day_increments

# 6. Simulation finale du Mouvement Brownien Géométrique (GBM)
M = S_0 * np.exp(drift + cumulated_bm)
```

In [487…

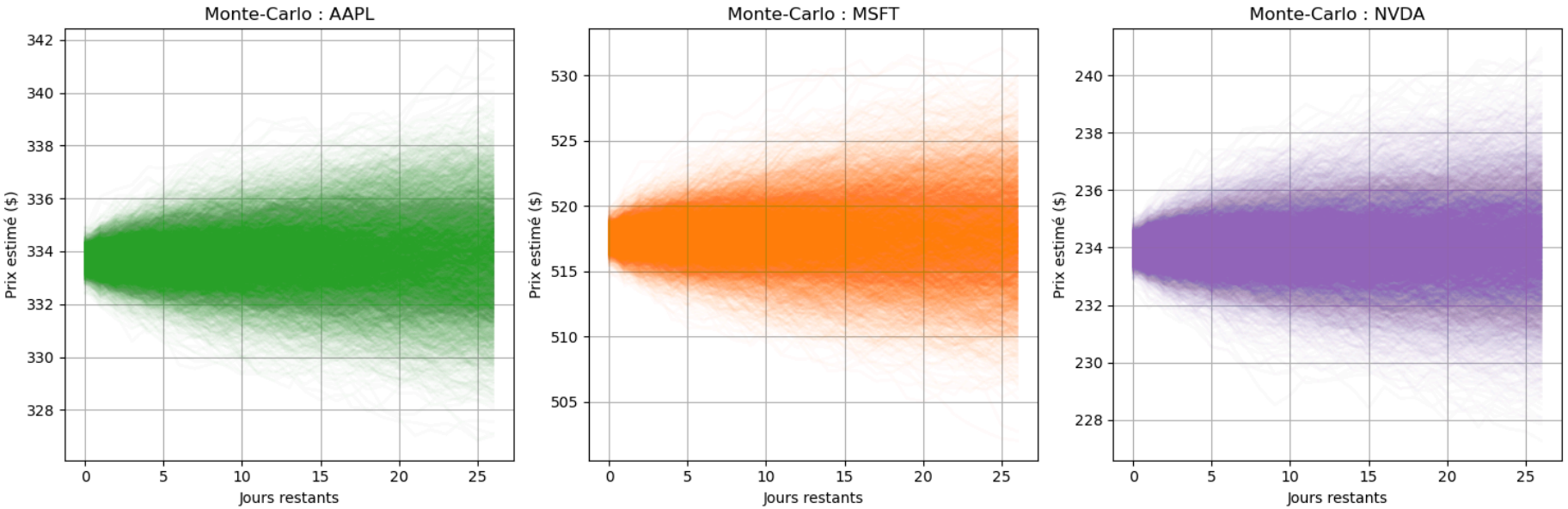
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
colors = ["#2ca02c", "#ff7f0e", "#9467bd"]

for i, (ax, ticker) in enumerate(zip(axes, tickers)):
  # .T permet d'avoir les 27 jours en abscisse et les 1000 trajectoires en ordonnée
  ax.plot(M[:, i, :].T, c=colors[i], alpha=0.015)

  # On utilise le titre pour le nom de l'actif au lieu de la légende
  ax.set_title(f"Monte-Carlo : {ticker}")
  ax.set_xlabel("Jours restants")
  ax.set_ylabel("Prix estimé ($)")
  ax.grid(True)
```

```
plt.tight_layout()
plt.show()
```



In [523…
```
# Récupérer les prix de tous les scénarios au dernier jour (jour 27)
prix_finaux = M[:, :, -1]

# Calculer l'écart-type des prix finaux pour chaque actif
risque_ecart_type = np.std(prix_finaux, axis=0)

for ticker, risk in zip(tickers, risque_ecart_type):
  print(f"Actif {ticker} - Volatilité du prix final (Écart-type): USD {risk:.2f}")
```

```
Actif AAPL - Volatilité du prix final (Écart-type): USD 1.88
Actif MSFT - Volatilité du prix final (Écart-type): USD 3.88
Actif NVDA - Volatilité du prix final (Écart-type): USD 1.90
```

In [524…
```
# Risque relatif en pourcentage par rapport au prix initial
risque_relatif = (risque_ecart_type / S_0.squeeze()) * 100

for ticker, risk in zip(tickers, risque_relatif):
  print(f"Actif {ticker} - Volatilité relative : {risk:.2f}%")
```

```
Actif AAPL – Volatilité relative : 0.56%
Actif MSFT – Volatilité relative : 0.75%
Actif NVDA – Volatilité relative : 0.81%
```

In [525…

```python
import numpy as np
from scipy.optimize import minimize

# 1. Définir la fonction objective à MINIMISER (Volatilité du portefeuille)
def portfolio_volatility(weights):
    return np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))

# 2. Contraintes et limites
# - La somme des poids doit être égale à 1 (100% du capital)
# - Chaque poids doit être entre 0 et 1 (pas de vente à découvert / short selling)
constraints = {'type': 'eq', 'fun': lambda w: np.sum(w) - 1.0}
bounds = [(0, 1) for _ in range(stocks_nb)]  # Généralisé pour 'stocks_nb' actifs

# Poids initiaux (équipondérés : 1/3 pour chaque actif)
init_guess = np.ones(stocks_nb) / stocks_nb

# 3. Lancer l'optimisation (Minimisation du risque)
resultat = minimize(
    portfolio_volatility,
    init_guess,
    method='SLSQP',
    bounds=bounds,
    constraints=constraints
)
poids_optimaux = resultat.x

# 4. Traduire en montant pour un capital de 100 000 €
capital = 100000
montants_optimaux = poids_optimaux * capital

print("--- Répartition optimale (Minimisation du risque) ---")
for ticker, poids, montant in zip(tickers, poids_optimaux, montants_optimaux):
    print(f"Actif {ticker} : {poids * 100:.2f}% du portefeuille soit {montant:,.2f} €")
```

```
--- Répartition optimale (Minimisation du risque) ---
Actif AAPL : 51.82% du portefeuille soit 51,818.40 €
Actif MSFT : 21.70% du portefeuille soit 21,702.30 €
Actif NVDA : 26.48% du portefeuille soit 26,479.30 €
```

In [526…

```python
# 1. Extraction des prix unitaires actuels (on retire les dimensions superflues de S_0)
prix_unitaires = S_0.squeeze()  # Forme (stocks_nb,)

# 2. Calcul du nombre d'actions entières à acheter
montants_optimaux = poids_optimaux * capital
nb_actions_entieres = np.floor(montants_optimaux / prix_unitaires)

# 3. Calcul du coût réel investi et du reste en cash
investissement_reel = nb_actions_entieres * prix_unitaires
capital_investi = np.sum(investissement_reel)
reste_cash = capital - capital_investi

# 4. Affichage détaillé par actif
print("--- Répartition en actions entières ---")
for ticker, nb, prix, montant_reel in zip(tickers, nb_actions_entieres, prix_unitaires, investissement_reel):
    print(f"Actif {ticker} : {int(nb)} actions achetées (à {prix:.2f} €) -> {montant_reel:,.2f} €")

print(f"\nCapital initial : {capital:,.2f} €")
print(f"Capital réellement investi : {capital_investi:,.2f} €")
print(f"Reste non investi (Cash) : {reste_cash:,.2f} €")
```

```
--- Répartition en actions entières ---
Actif AAPL : 155 actions achetées (à 333.69 €) -> 51,721.95 €
Actif MSFT : 41 actions achetées (à 517.53 €) -> 21,218.73 €
Actif NVDA : 113 actions achetées (à 233.95 €) -> 26,436.35 €

Capital initial : 100,000.00 €
Capital réellement investi : 99,377.03 €
Reste non investi (Cash) : 622.97 €
```

In [527…

```python
import matplotlib.pyplot as plt
import numpy as np

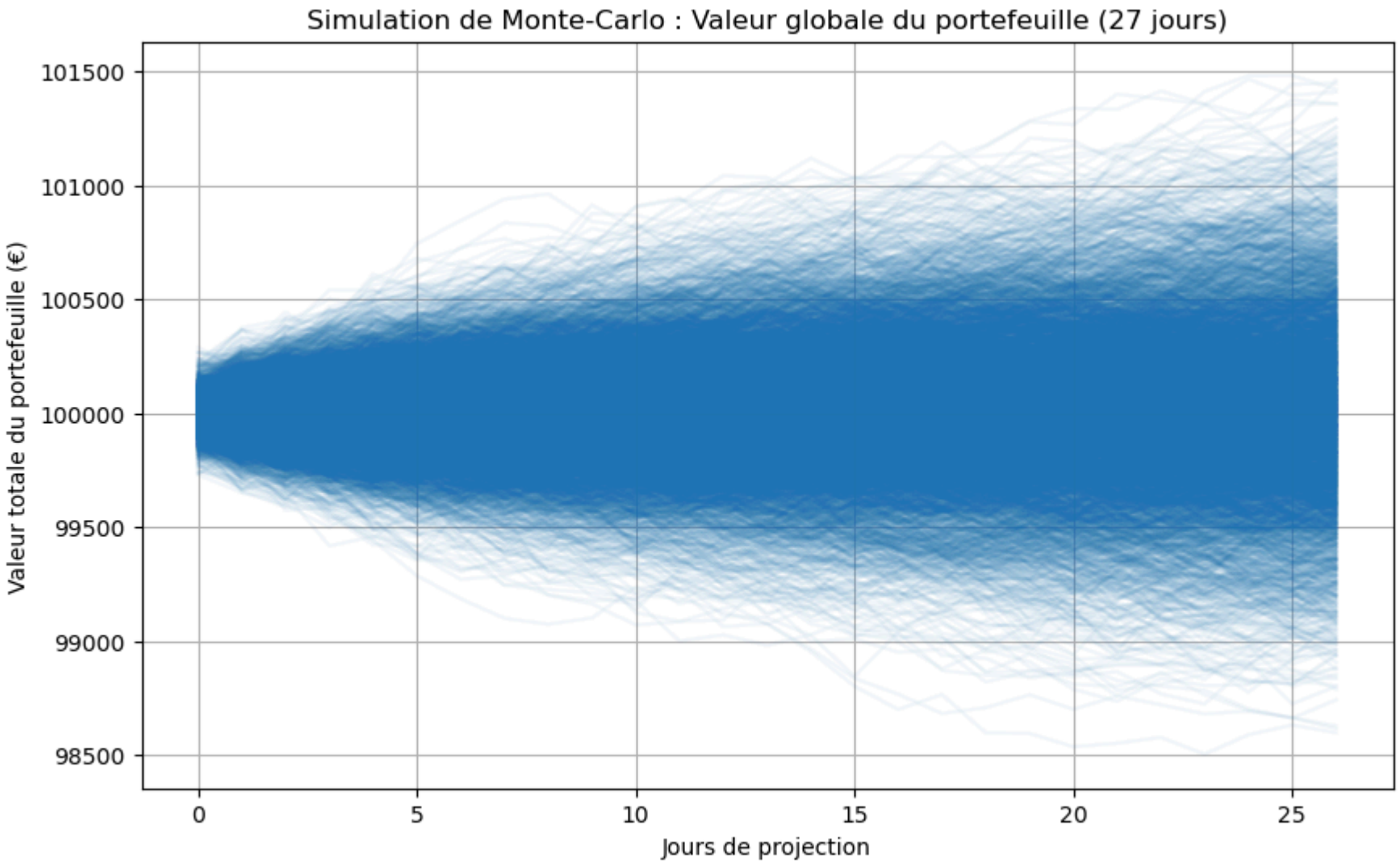
# 1. Calcul de la valeur de chaque ligne pour tous les scénarios sur les 27 jours
# M a pour forme : (n_samples, stocks_nb, days_left) -> (5000, 3, 27)
# nb_actions_entieres a pour forme : (3,) qu'on reshape en (1, stocks_nb, 1) pour le broadcasting
```

```python
valeur_lignes = M * nb_actions_entieres.reshape(1, stocks_nb, 1)

# 2. Somme sur les 3 actifs (axis=1) pour obtenir la valeur des actions par scénario
# valeur_actions_totale aura pour forme : (5000, 27)
valeur_actions_totale = np.sum(valeur_lignes, axis=1)

# On ajoute le reste_cash (qui reste constant sur toute la période pour chaque scénario)
# reshape(1, 1) permet d'ajouter le cash à chaque jour et pour chaque échantillon
portefeuille_valeurs = valeur_actions_totale + reste_cash

# 3. Tracé du "cône" de Monte-Carlo pour le portefeuille global
plt.figure(figsize=(10, 6))
plt.plot(
    portefeuille_valeurs[:, :].T, color="#1f77b4", alpha=0.05
)  # .T pour avoir les 27 jours en abscisse
plt.title(
    "Simulation de Monte-Carlo : Valeur globale du portefeuille (27 jours)"
)
plt.xlabel("Jours de projection")
plt.ylabel("Valeur totale du portefeuille (€)")
plt.grid(True)
plt.show()
```



In [528…

```python
import numpy as np

# 1. Valeur initiale globale du portefeuille (au jour 0)
# Toutes les trajectoires partent de la même valeur initiale (capital investi + cash)
valeur_initiale = portefeuille_valeurs[0, 0]  # ou on peut utiliser directement la variable 'capital'

# 2. Valeurs finales du portefeuille pour les 5000 scénarios au dernier jour (jour 27)
```

```
valeurs_finales = portefeuille_valeurs[:, -1]

# 3. Calcul des profits et pertes (PnL) en euros
pnl = valeurs_finales - valeur_initiale

# 4. Calcul de la VaR 95% (le 5e percentile des PnL)
# Cela signifie qu'il y a 5 % de chance de faire pire que cette perte sur l'horizon
var_95_absolue = np.percentile(pnl, 5)

# 5. Affichage des résultats
print(f"Valeur initiale du portefeuille : {valeur_initiale:,.2f} €")
print(f"Pire perte estimée (VaR 95%) : {-var_95_absolue:,.2f} €")
print(f"Seuil de valeur minimale du portefeuille à 95% de confiance : {valeur_initiale + var_95_absolue:,.2f} €")
```

```
Valeur initiale du portefeuille : 99,952.13 €
Pire perte estimée (VaR 95%) : 612.90 €
Seuil de valeur minimale du portefeuille à 95% de confiance : 99,339.23 €
```

In [529…

```
import numpy as np

# 1. Espérance de gain en euros (moyenne des PnL sur tous les scénarios)
esperance_gain_euros = np.mean(pnl)

# 2. Espérance de rendement en pourcentage
rendements_scenarios = (valeurs_finales - valeur_initiale) / valeur_initiale
esperance_rendement_pct = np.mean(rendements_scenarios) * 100

# 3. Affichage des résultats
print(f"Valeur initiale du portefeuille : {valeur_initiale:,.2f} €")
print(f"Espérance de gain (en €) : {esperance_gain_euros:+,.2f} €")
print(f"Espérance de rendement (en %) : {esperance_rendement_pct:+.2f} %")
```

```
Valeur initiale du portefeuille : 99,952.13 €
Espérance de gain (en €) : +69.05 €
Espérance de rendement (en %) : +0.07 %
```

In [530…

```
# 1. Nombre de scénarios avec un PnL positif vs négatif
nb_scenarios_gagnants = np.sum(pnl > 0)
nb_scenarios_perdants = np.sum(pnl < 0)

# 2. En pourcentage sur les 5000 scénarios
```

```
prob_gain = (nb_scenarios_gagnants / n_samples) * 100
prob_perte = (nb_scenarios_perdants / n_samples) * 100

print(f"Probabilité de faire un gain : {prob_gain:.2f}% ({nb_scenarios_gagnants} scénarios)")
print(f"Probabilité de faire une perte : {prob_perte:.2f}% ({nb_scenarios_perdants} scénarios)")
```

```
Probabilité de faire un gain : 56.04% (2802 scénarios)
Probabilité de faire une perte : 43.96% (2198 scénarios)
```

In [534…

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import yfinance as yf

# 1. Calcul des vrais rendements journaliers historiques de vos actifs
returns_historiques = data.pct_change().dropna()
capital_depart = 100000

# 2. Portefeuille Optimal (Min Vol)
rendements_portefeuille_opt = np.dot(returns_historiques.values, poids_optimaux)
portefeuille_historique = pd.Series(
    capital_depart * (1 + rendements_portefeuille_opt).cumprod(),
    index=returns_historiques.index,
)

# 3. Portefeuille Équipondéré (1/3 chacun)
poids_equi = np.ones(stocks_nb) / stocks_nb
rendements_portefeuille_equi = np.dot(returns_historiques.values, poids_equi)
portefeuille_equi_historique = pd.Series(
    capital_depart * (1 + rendements_portefeuille_equi).cumprod(),
    index=returns_historiques.index,
)

# 4. Récupération et calcul du S&P 500 sur la même période exacte
date_debut = returns_historiques.index[0]
date_fin = returns_historiques.index[-1]
sp500_data = yf.download(
    "^GSPC", start=date_debut, end=date_fin, progress=False, auto_adjust=True
)["Close"]

if isinstance(sp500_data, pd.DataFrame):
```

```
  sp500_data = sp500_data.iloc[:, 0]

returns_sp500 = sp500_data.pct_change().dropna()

# Synchronisation stricte des dates avec les actifs pour un comparatif propre
returns_commun = returns_historiques.join(returns_sp500, how="inner")
dates_communes = returns_commun.index

# Recalcul synchronisé des portefeuilles sur les mêmes dates communes
rend_opt_sync = np.dot(
    returns_commun.iloc[:, :-1].values, poids_optimaux
)
rend_equi_sync = np.dot(returns_commun.iloc[:, :-1].values, poids_equi)
rend_sp500_sync = returns_commun.iloc[:, -1].values

portefeuille_historique = pd.Series(
    capital_depart * (1 + rend_opt_sync).cumprod(), index=dates_communes
)
portefeuille_equi_historique = pd.Series(
    capital_depart * (1 + rend_equi_sync).cumprod(), index=dates_communes
)
portefeuille_sp500 = pd.Series(
    capital_depart * (1 + rend_sp500_sync).cumprod(), index=dates_communes
)

# 5. Affichage graphique du backtest comparatif
plt.figure(figsize=(12, 6))
plt.plot(
    portefeuille_historique.index,
    portefeuille_historique,
    label="Portefeuille Optimal (Min Vol)",
    color="blue",
    linewidth=2,
)
plt.plot(
    portefeuille_equi_historique.index,
    portefeuille_equi_historique,
    label="Portefeuille Équipondéré",
    color="orange",
    linestyle="--",
    linewidth=1.5,
```

```
)
plt.plot(
    portefeuille_sp500.index,
    portefeuille_sp500,
    label="S&P 500 (Benchmark)",
    color="green",
    linestyle=":",
    linewidth=1.5,
)

plt.title(
    "Backtest Historique : Portefeuille Optimal vs Équipondéré vs S&P 500"
)
plt.xlabel("Date")
plt.ylabel("Valeur du portefeuille (€)")
plt.legend()
plt.grid(True)
plt.show()

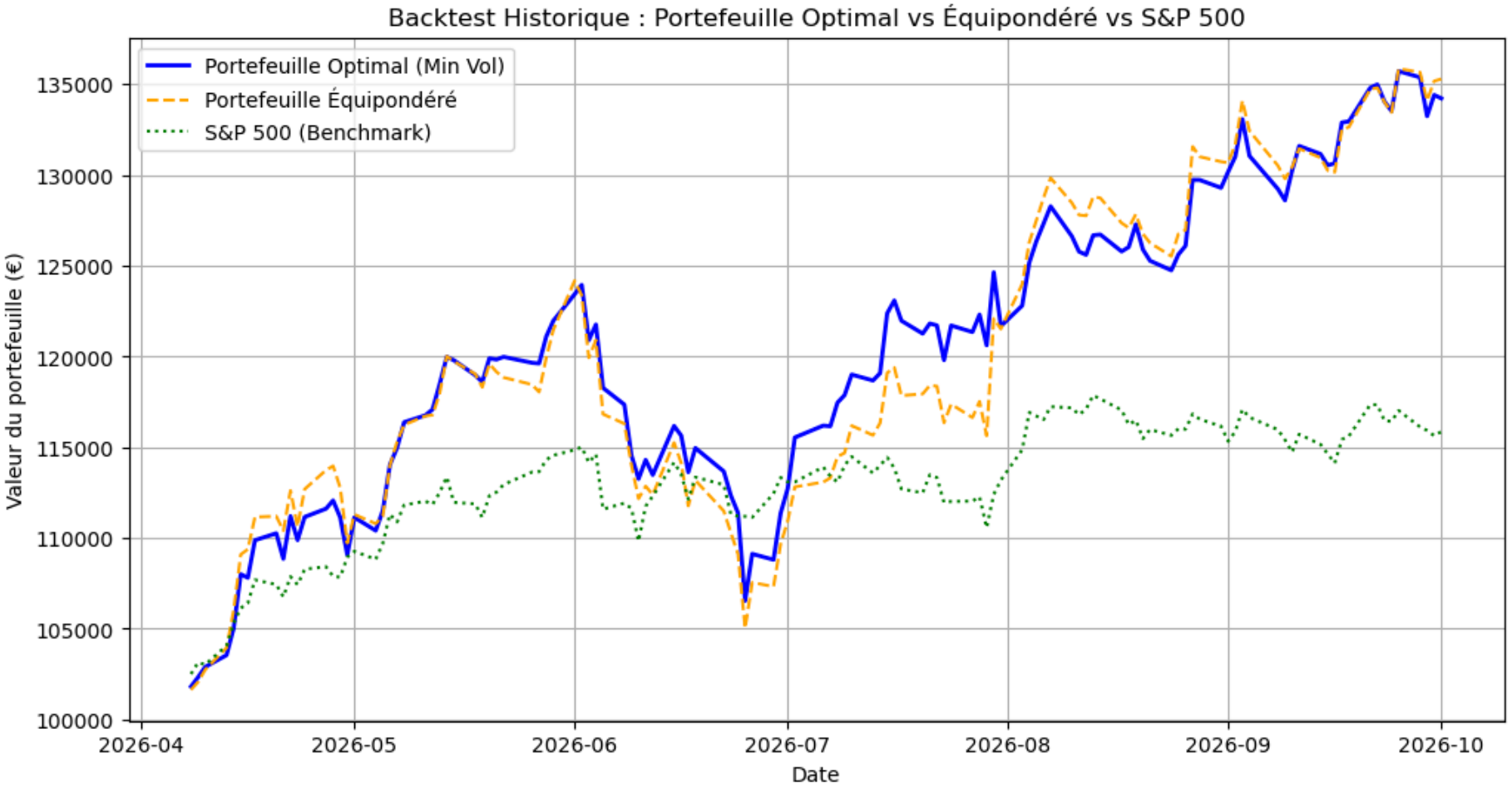
# 6. Calcul des métriques et du Ratio de Sharpe
taux_sans_risque_annuel = 0.02
taux_sans_risque_journalier = (1 + taux_sans_risque_annuel) ** (
    1 / 252
) - 1


def calculer_metriques(rendements, serie_valeur):
  perf = (serie_valeur.iloc[-1] - capital_depart) / capital_depart * 100
  vol = np.std(rendements) * np.sqrt(252) * 100
  sharpe = (
      np.mean(rendements - taux_sans_risque_journalier) / np.std(rendements)
  ) * np.sqrt(252)
  return perf, vol, sharpe


perf_opt, vol_opt, sharpe_opt = calculer_metriques(
    rend_opt_sync, portefeuille_historique
)
perf_equi, vol_equi, sharpe_equi = calculer_metriques(
    rend_equi_sync, portefeuille_equi_historique
)
```

```
perf_sp500, vol_sp500, sharpe_sp500 = calculer_metriques(
    rend_sp500_sync, portefeuille_sp500
)

print("--- Bilan Comparatif Final du Backtest ---")
print(
    f"Portefeuille Optimal  -> Perf : {perf_opt:+.2f}% | Volatilité :"
    f" {vol_opt:.2f}% | Sharpe : {sharpe_opt:.2f}"
)
print(
    f"Portefeuille Équipondéré -> Perf : {perf_equi:+.2f}% | Volatilité"
    f" : {vol_equi:.2f}% | Sharpe : {sharpe_equi:.2f}"
)
print(
    f"S&P 500 (Benchmark)   -> Perf : {perf_sp500:+.2f}% | Volatilité"
    f" : {vol_sp500:.2f}% | Sharpe : {sharpe_sp500:.2f}"
)
```




```
--- Bilan Comparatif Final du Backtest ---
Portefeuille Optimal  -> Perf : +34.23% | Volatilité : 20.32% | Sharpe : 2.98
Portefeuille Équipondéré -> Perf : +35.30% | Volatilité : 21.82% | Sharpe : 2.86
S&P 500 (Benchmark)   -> Perf : +15.86% | Volatilité : 12.48% | Sharpe : 2.32
```